ABSTRACT

Rapid urbanization and the growing demand for instant delivery have made quick commerce an important part of urban retail in India. Running it well depends on three linked decisions, namely where the dark stores are placed, which customer zones each store serves and how much of each product it holds. This project places these decisions in one binary optimization model that respects storage capacity, customer demand, a delivery-time promise and an opening-cost budget. The model is written as a Quadratic Unconstrained Binary Optimization (QUBO) problem and solved with the Quantum Approximate Optimization Algorithm (QAOA) on a Qiskit statevector simulator, and every answer is compared with an exact Mixed Integer Linear Programming (MILP) baseline built with PuLP and solved by CBC. Both answers are checked by an independent validator that re-derives every constraint from the raw data.

The system is implemented as a FastAPI backend with a Supabase PostgreSQL database and a React dashboard. Its data layer holds 48 candidate retail outlets and 163 demand zones taken from OpenStreetMap, 7,824 store-to-zone road travel times from OSRM, 25 products from a Blinkit catalogue sample and a population-derived demand estimate. On the prototype instance of three stores, three areas and three products (11 qubits), QAOA with depth p = 3 reached the proven optimum of ₹14,716.14, the same value found by PuLP/CBC and by exhaustive enumeration of all 2,048 bit strings, with 100% of demand served and no constraint violation. On a separate 15-qubit instance built from the OpenStreetMap data (four candidate outlets, four zones, 25 products), both methods reached ₹100,704.01, but only 50% of demand could be served, because two of the four zones lie outside the 15-minute delivery promise of every selected store. A later Supabase-backed run recorded by the project team gave ₹101,374.45 for both methods. These are different instances and are reported separately. In every case CBC was faster by two orders of magnitude, and lower circuit depths returned feasible but sub-optimal plans. The results show that the joint model is correct and can be executed on a gate-based circuit, not that QAOA outperforms classical optimization. Demand is a population-derived estimate, store economics are prototype assumptions and the candidate outlets are not verified dark stores.

Keywords: Quick Commerce, Dark Stores, Facility Location, Inventory Allocation, QUBO, QAOA, Qiskit, Mixed Integer Linear Programming, PuLP, FastAPI.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Background: Quick Commerce in India 1

1.2 Dark Stores and the Delivery-Time Promise 1

1.3 Inventory Decisions in a Dark Store 2

1.4 Motivation for Joint Optimization 2

1.5 Quantum Optimization as a Candidate Method 2

1.6 Problem Statement 3

1.7 Objectives of the Project 3

1.8 Scope and Limitations 4

1.9 Tools and Environment 4

1.10 Organisation of the Report 4

CHAPTER 2 LITERATURE SURVEY 5

2.1 Overview of the Survey 5

2.2 Base Paper 6

2.3 Facility-Location Optimization 6

2.4 Inventory and Warehouse Optimization 7

2.5 QUBO Models of Combinatorial Problems 7

2.6 The Quantum Approximate Optimization Algorithm 8

2.7 Classical Optimization Approaches 8

2.8 Hybrid Quantum-Classical Pipelines 8

2.9 Summary of Limitations and Research Gap 9

CHAPTER 3 SYSTEM ANALYSIS 10

3.1 Existing System 10

3.2 Limitations of the Existing System 11

3.3 Proposed System 12

3.4 Feasibility Study 14

3.5 Hardware and Software Requirements 15

3.6 Functional Requirements 16

3.7 Non-Functional Requirements 16

3.8 System Constraints and Assumptions 17

3.9 System Architecture 17
CHAPTER 4 SYSTEM DESIGN AND METHODOLOGY 19
4.1 Design Overview 19
4.2 Datasets and Provenance 19
4.3 Conceptual Formulation (Conference Paper) 20
4.4 Implemented Optimization Model 21
4.5 QUBO Formulation 24
4.6 Worked Example on the Prototype Instance 26
4.7 Ising Mapping and the QAOA Procedure 27
4.8 Classical Baseline 29
4.9 Evaluation Measures 29
4.10 Flow Charts 30
CHAPTER 5 MODULES 32
5.1 Module 1: Data Collection and Preprocessing 32
5.2 Module 2: Model and QUBO Formulation 33
5.3 Module 3: QAOA Optimization 33
5.4 Module 4: Solution Validation and Results 34
5.5 Module 5: Classical MILP Baseline 34
5.6 Module 6: Planner Interface 35
CHAPTER 6 IMPLEMENTATION 36
6.1 Backend Structure 36
6.2 Database Integration 36
6.3 REST API 37
6.4 QAOA Implementation 38
6.5 QUBO Construction 38
6.6 Classical Baseline Implementation 39
6.7 Frontend Dashboard 39
6.8 Configuration and Deployment 39
CHAPTER 7 TESTING 40
7.1 Testing Strategy 40
7.2 Test Environment 40

7.3 Test Suite Summary 40
7.4 Solver and Constraint-Validation Tests 41
7.5 Test Execution Results 41
7.6 Design-Level Test Cases 41
7.7 Checks on the Experimental Solutions 42
CHAPTER 8 RESULTS AND DISCUSSION 43
8.1 Experimental Setup 43
8.2 Overview of the Experiments 43
8.3 Experiment A: Prototype Instance 43
8.4 Sensitivity to Circuit Depth 45
8.5 Experiment B: Supabase-Backed Instance 46
8.6 Experiment C: OpenStreetMap Instance with 25 Products 46
8.7 Experiment D: Full Stored Network (Classical Only) 48
8.8 Exploratory Scaling Study 48
8.9 Qubit Requirements and Simulation Memory 49
8.10 Comparison of the Classical and Quantum Approaches 50
8.11 Comparison with Earlier Studies 51
8.12 Discussion 51
8.13 Limitations 51
CHAPTER 9 CONCLUSION AND FUTURE SCOPE 53
9.1 Conclusion 53
9.2 Contributions 53
9.3 Future Scope 54
REFERENCES 55

LIST OF FIGURES

Figure 3.1 Existing sequential planning compared with the proposed joint planning 10

Figure 3.2 Architecture and data flow of the implemented system 18

Figure 3.3 Three-layer architecture of the planning platform (design) 18

Figure 4.1 Modular workflow of the QAOA framework for joint store location and inventory optimization 19

Figure 4.2 QAOA workflow as implemented in the backend 29

Figure 4.3 Flow chart of the QAOA-based framework for joint store location and inventory optimization 30

Figure 4.4 Flow chart of the fast path for live order handling (design, not implemented) 31

Figure 4.5 Flow chart of the slow path for re-optimization 31

Figure 5.1 Block diagram of Module 1 (data collection and preprocessing) 32

Figure 5.2 Block diagram of Module 2 (QUBO model formulation) 33

Figure 5.3 Block diagram of Module 4 (solution validation and results) 34

Figure 5.4 Block diagram of Module 5 (classical MILP baseline) 34

Figure 5.5 Block diagram of Module 6 (planner interface with fast and slow paths) 35

Figure 6.1 Main database tables and relationships 37

Figure 8.1 Objective value of QAOA and classical MILP on the prototype instance (Experiment A) 44

Figure 8.2 Service level and resource utilisation of the prototype solution (Experiment A) ... 45

Figure 8.3 QAOA objective on the prototype instance for different depths and optimizer budgets 46

Figure 8.4 Re-run wall-clock time of QAOA and CBC in Experiments A and C (log scale) .. 47

Figure 8.5 Decision variables against the number of demand zones for 48 candidate stores (upper bound, all links feasible) 50

LIST OF TABLES

Table 2.1 Comparative literature survey of QAOA-based optimization studies 5

Table 3.1 Implementation status of the main features 13

Table 3.2 Existing system compared with the proposed system 13

Table 3.3 Software requirements of the implemented system 15

Table 3.4 Hardware used for development and experiments 15
Table 3.5 Functional requirements supported by the implementation 16
Table 3.6 Non-functional requirements and how they are met 16
Table 3.7 Constraints and assumptions of the model and system 17
Table 4.1 Datasets used in the study and their provenance 19
Table 4.2 Variables and parameters of the implemented model 22
Table 4.3 Conceptual formulation compared with the implemented model 24
Table 4.4 Derived data of the prototype instance (budget B = ₹20,000, promise 15 minutes) 26
Table 5.1 Inputs, tools and outputs of the modules 35
Table 6.1 Main backend files and their responsibilities 36
Table 6.2 REST API endpoints of the backend 37
Table 7.1 Automated test suite of the backend 40
Table 7.2 Design-level test cases and their coverage 41
Table 7.3 Validator checks on the experimental solutions 42
Table 8.1 Experiments reported in this chapter 43
Table 8.2 Results of Experiment A (prototype instance) 44
Table 8.3 Effect of circuit depth and optimizer budget on the prototype instance (re-run) 45
Table 8.4 Results of Experiment C (4 OSM outlets × 4 zones × 25 products) 47
Table 8.5 Results of Experiment D (classical only, 48 outlets × 46 zones × 25 products) 48
Table 8.6 Recorded results of the exploratory scaling study (earlier model, synthetic instances) 49
Table 8.7 Qubits and statevector memory for the instances of this study 49
Table 8.8 Comparison of the classical MILP and the QAOA approach 50
Table 8.9 Problem sizes in earlier studies and in the present work 51

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CBC	COIN-OR Branch and Cut (open-source MILP solver)
COBYLA	Constrained Optimization BY Linear Approximation
CRUD	Create, Read, Update, Delete
FK / PK	Foreign key / Primary key
MILP	Mixed Integer Linear Programming
NISQ	Noisy Intermediate-Scale Quantum
ORM	Object-Relational Mapper
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
QAOA	Quantum Approximate Optimization Algorithm
QUBO	Quadratic Unconstrained Binary Optimization
REST	Representational State Transfer
SKU	Stock-Keeping Unit
UFLP	Uncapacitated Facility Location Problem
VQE	Variational Quantum Eigensolver

CHAPTER 1
INTRODUCTION

1.1 Background: Quick Commerce in India

Quick commerce, also called q-commerce, is a retail model in which groceries and daily essentials reach the customer within a few minutes of the order, usually between ten and thirty. In India it has grown very fast and has changed the way many city households buy milk, vegetables, snacks and other everyday items. Industry estimates put the number of dark stores in the country at more than five thousand, spread over more than four hundred cities. The rise of Indian hyperlocal delivery shows how quickly the model has scaled and how much it depends on a dense network of fulfilment points close to the customer [28].

The speed promise changes the economics of retail. In a conventional supermarket the customer travels to the store, so the retailer competes on assortment and price. In quick commerce the retailer travels to the customer, so every minute between the store and the doorstep is a cost that the company pays directly through rider time, and a promise that it has to keep to retain the customer. Placement, service area and stock therefore become operational decisions that have to be revisited as demand grows and shifts across a city.

1.2 Dark Stores and the Delivery-Time Promise

A dark store is a small warehouse that is closed to the public and used only for picking and packing online orders. There is no shop floor and no billing counter. Because the customer expects the order in about ten minutes, the location of the store matters more than almost anything else; a customer who is promised ten minutes will not accept fifteen. A store that is placed even a little away from its demand zones makes every delivery slower, and a store that runs out of a popular item loses the sale straight away. The business is therefore decided by where the stores are and what is kept inside them.

The delivery-time promise acts as a hard service radius around each store. A zone that cannot be reached within the promise is not a poor option for that store but no option at all. Road travel time, not straight-line distance, is the right measure for this radius, because rivers, one-way streets and the shape of the road network can make two equally distant zones very different in practice. This project therefore uses routed road travel times from the Open Source Routing Machine (OSRM) over OpenStreetMap road geometry [18], [12] wherever they are available.

1.3 Inventory Decisions in a Dark Store

A dark store has a fixed storage volume, typically a few thousand units of shelf space, and it must carry the stock needed for every zone it serves. Stocking more products raises availability but fills the shelves and ties up working capital, while stocking less risks stock-outs. Classical inventory theory, starting from the economic order quantity of Harris [13], treats this as a trade-off between ordering, holding and shortage costs at a single location. In a dark-store network the same trade-off is coupled to location, because the stock a store must hold depends on which zones are assigned to it, and the zones it can serve depend on where it is.

1.4 Motivation for Joint Optimization

Three planning choices have to be made together. A company must decide which candidate locations should be opened, which customer zones each opened store should serve, and which products, and how much of each, every store should hold. Opening more stores shortens delivery trips but raises rent and fixed cost. In practice planners often fix the locations first and plan the inventory afterwards, so a good site can end up with a poor stock mix and a well stocked store can end up serving the wrong neighbourhood. Studies on micro-fulfilment network planning in quick commerce report that placement and service speed are closely linked [30], and work on dark store portfolios points out that a network becomes fragile when these choices are made without regard to disruption [27]. Joint location-inventory models in the operations research literature make the same argument for conventional distribution networks [6], [29].

The difficulty with this kind of planning is the way the choices multiply. Every candidate location can be opened or kept closed, and every zone can be assigned to any store that reaches it within the promise. With 48 candidate locations there are already 2^{48} opening patterns before a single zone is assigned. A planner cannot check such a number of combinations by hand, and a method that handles the choices one after the other cannot be sure that the final combination is a good one. This is the reason for treating the decisions as one optimization problem.

1.5 Quantum Optimization as a Candidate Method

Quantum computing works on qubits. Unlike a classical bit, a qubit can be in a superposition of 0 and 1, and a register of n qubits is described by 2^n complex amplitudes. Gates change this state, and a measurement returns one bit string with a probability that depends on the final state. The devices available today are small and noisy, so only shallow circuits can be run on them with reasonable reliability [26]. For this reason, algorithms that combine a short

parameterized quantum circuit with a classical optimizer are the main choice for optimization on such devices [24], and QAOA is one of them [7].

Facility location and allocation problems can be written as quadratic binary models [17], [9], and QAOA is designed to search such a model with a shallow circuit whose parameters are tuned by a classical optimizer. Recent studies have written facility location and load balancing as QUBO problems and solved them with QAOA [21], and QAOA has also been applied to vehicle routing [3], [8]. A broad mapping study of quantum and quantum-inspired methods in operations research confirms that logistics is one of the most active application areas, while also noting that much of this work is still at the proof-of-concept stage [10]. This project adopts the same cautious position: QAOA is studied as a candidate method whose correctness and limits are measured against an exact classical solver.

1.6 Problem Statement

Given a set of candidate store locations with opening costs and storage capacities, a set of demand zones with product-level demand and a delivery-time promise, a product catalogue with unit volumes and costs, road travel times between every store and zone, and an opening-cost budget, the problem is to decide which stores to open and which zones each open store serves, so that the net operating margin is maximised while each zone is served by at most one store, only open stores serve zones, the stock required at each store fits its capacity, the total opening cost stays within the budget and every served zone is reached within its promise. The stock of every product at every store follows from these decisions.

1.7 Objectives of the Project

1. To formulate store selection, zone assignment and inventory allocation as one binary optimization model with explicit capacity, budget, single-sourcing and delivery-time constraints.
2. To encode the model as a QUBO with as few qubits as possible and to solve it with QAOA on a Qiskit statevector simulator.
3. To solve the identical instance exactly with a PuLP/CBC MILP baseline and to validate both answers independently of either solver.
4. To build a working planning platform with data ingestion from open sources, a FastAPI backend, a PostgreSQL (Supabase) database and a React dashboard.
5. To report objective value, service level, utilisation, constraint violations, runtime and optimality gap honestly, including the cases in which QAOA falls short.

1.8 Scope and Limitations

The study area is Puducherry, the planning is done for a single period, and the demand is an estimate based on population, because no public source of quick-commerce order data exists for the city. The candidate stores are real retail outlets mapped in OpenStreetMap, not verified operating dark stores, and their opening costs and capacities are prototype assumptions. The quantum part is run on a noiseless statevector simulator, not on quantum hardware, so the results describe how the formulation behaves in the ideal case and not how it would behave on a real device today. These limits are stated here so that the results in the later chapters are read in the right light.

1.9 Tools and Environment

The optimization logic is written in Python. Qiskit 2.5 and Qiskit Optimization 0.7 are used to build the quadratic program, convert it to an Ising operator and construct the QAOA circuit, which is executed with Qiskit's exact `StatevectorSampler` primitive [14]. SciPy provides the COBYLA optimizer [25] that tunes the circuit angles, and NumPy holds the QUBO matrix. The MILP baseline is written with PuLP 2.9 and solved with its bundled CBC solver [20]. The backend uses FastAPI and Uvicorn with SQLAlchemy 2.1 as the object-relational mapper, connected to a Supabase-hosted PostgreSQL database. The dashboard is written in React 19 with Vite, React Router, axios, React-Leaflet for maps and Recharts for charts. Data are collected from the Overpass API, OSRM and Wikidata with the Requests package.

1.10 Organisation of the Report

The report is organised in nine chapters. Chapter 2 surveys earlier work on facility location, inventory optimization, QUBO models, QAOA and hybrid quantum-classical methods. Chapter 3 analyses the existing approach and the proposed system, with the feasibility study, requirements, constraints and architecture. Chapter 4 presents the design and methodology, including the mathematical formulation, the QUBO encoding and the QAOA procedure. Chapter 5 describes the six modules of the system, Chapter 6 the implementation, Chapter 7 the testing, and Chapter 8 the experimental results and their discussion. Chapter 9 concludes the report and outlines the future scope.

CHAPTER 2
LITERATURE SURVEY

2.1 Overview of the Survey

Fifteen studies on quantum and quantum-inspired optimization for inventory, warehouse and facility problems were reviewed for this work, together with the classical and methodological literature on which the formulation rests. Table 2.1 gives the dataset, the publisher, the technique and the limitation reported for each of the fifteen studies. Most of them use QAOA or a hybrid of QAOA with a classical solver, and almost all of them stop at small instances.

Table 2.1 Comparative literature survey of QAOA-based optimization studies

No.	Author(s) & Year	Dataset	Publisher / Year	Techniques Used	Limitations
1	Sinha (2026)	Synthetic multi-warehouse inventory nodes (30 items, 5 locations)	LinkedIn Core Insights / 2026	QAOA via IBM Qiskit (QUBO formulation) vs. classical PuLP	Limited to static penalty bounds; vulnerable to local minima if constraints are not balanced.
2	Vanderbist et al. (2026)	Simulated benchmark maritime/logistics shipping networks	IEEE / 2026	Hybrid quantum-classical alternating QAOA	Significant classical parameter-tuning overhead for deeper layers ($p \geq 3$).
3	Afrasiabi et al. (2026)	Industrial logistics framework logs and cross-docking data	MDPI / 2026	Hybrid QAOA + VQE + quantum annealing	Hardware error-mitigation overhead increases processing time sharply at scale.
4	Li & Integrated Systems (2026)	Real-world omnichannel warehouse fulfilment data matrices	ResearchGate / 2026	Hybrid classical-quantum spin Hamiltonian	High penalty-factor sensitivity can give physically infeasible solutions.
5	Tokar (2026)	Meta-analysis data across 10+ logistics journals	ResearchGate / 2026	Systematic literature review and content mapping	Qualitative mapping without execution metrics for hardware-level quantum gates.
6	Tata Elxsi Pilot (2025)	Proprietary automotive spare-parts inventory (30 critical parts)	Tata Elxsi Insights / 2025	QAOA executed on native IBM quantum hardware	Limited to selecting 5 of 30 parts by hardware gate-fidelity thresholds.
7	Vinci et al. (2025)	OR-Library facility location benchmarks	ScienceDirect / 2025	Compact unbalanced-penalisation QAOA	Accuracy degrades under hardware-calibrated noise for larger instances.
8	Nishimura et al. (2025)	Simulated multi-item warehouse rack-positioning layouts	arXiv / 2025	Hybrid quantum-classical solver with QUBO mapping	Restricted to shallow depths ($p = 1$ or 2) by noisy gates.
9	Zalando Engineering (2025)	Large-scale e-commerce SKU-level distribution matrices	Zalando Blog / 2025	Deep forecasting + quantum-informed network paths	Conceptual integration framework without full quantum execution.
10	Merali et al. (2025)	SME supply disruptions and multi-echelon network records	GRIP / 2025	Cloud-based hybrid QAOA platforms	High latency between classical edge optimizers and remote quantum back ends.

No.	Author(s) & Year	Dataset	Publisher / Year	Techniques Used	Limitations
11	Ullah et al. (2024)	Multi-variable warehouse capacity and lead-time constraints	ResearchGate / 2024	Multi-variable QAOA	Quadratic growth of variables when soft slack terms are added for many products.
12	Ganz et al. (2024)	Uncapacitated facility location benchmark	arXiv / 2024	Gate-based QAOA vs. quantum annealing	No testing on multi-product or multi-period inventory control.
13	Wang et al. (2023)	Standard UFLP testbeds	arXiv / 2023	Variational algorithm preserving the feasible space (VQA-PFS)	Specialised ansatz that is difficult to adapt to multi-period inventory.
14	Hedayatnia et al. (2023)	Three-echelon base-level spare support records	MDPI / 2023	Uncertainty theory + quantum-inspired metaheuristics	Fully emulated classically; no tests on physical qubits.
15	Bhattacharyya et al. (2022)	Multi-node allocation coordinate profiles	ResearchGate / 2022	Ising-Hamiltonian QAOA through the Qiskit API	Performance drops sharply beyond single-digit supply nodes.

2.2 Base Paper

The first entry of Table 2.1, Sinha (2026), is taken as the base paper of this project. The study writes a multi-warehouse inventory problem as a QUBO, solves it with QAOA in Qiskit and compares the answer with a classical PuLP model on a small synthetic network of thirty items and five locations. This is the same combination of tools that is used in the present work. The limitation reported in Table 2.1 is that the penalty bounds are fixed, so the search falls into local minima when the constraints are not balanced. This is why the penalty weight in the proposed model is derived from the data of each instance, and why a classical baseline is kept next to the quantum solver.

The present work differs from the base paper in three ways. Sinha (2026) works on inventory only, with synthetic data. This project adds the siting of the stores and the assignment of the zones to the same model, uses open data for a real city, and takes the road travel time as the measure of delivery.

2.3 Facility-Location Optimization

Facility location is one of the oldest problems in operations research. Reviews by Owen and Daskin [22] and by Melo, Nickel and Saldanha-da-Gama [19] trace its development from single-period fixed-charge models to capacitated, multi-period and supply-chain-integrated variants, and they note that the capacitated, single-sourcing version is NP-hard. The maximal covering location problem of Church and ReVelle [5] introduced the idea of serving demand only within a service distance, which is the classical counterpart of the delivery-time promise used here. More recent classical studies have moved towards uncertainty, with learning-

augmented mechanisms that use demand predictions to improve placement [1], and towards integrated models that combine facility location with order allocation across two supply-chain levels [2].

On the quantum side, Vinci et al. (2025) proposed a compact QAOA with unbalanced penalisation and tested it on OR-Library benchmarks, where the accuracy fell sharply on the larger instances once hardware-calibrated noise was added. Ganz et al. (2024) compared gate-based QAOA with quantum annealing on an uncapacitated facility location benchmark, but did not consider multi-product or multi-period inventory. Wang et al. (2023) designed a variational algorithm that keeps the search inside the feasible space for uncapacitated facility location, although the ansatz is so specialised that it is difficult to adapt to inventory settings. The work of Moncayo-Martínez and He [21] writes facility location and load balancing as QUBO problems and solves them with QAOA, and it is the closest formulation to ours.

2.4 Inventory and Warehouse Optimization

Joint location-inventory models were developed classically by Daskin, Coullard and Shen [6] and by Shen, Coullard and Daskin [29], who showed that inventory cost changes the optimal number and position of facilities. These studies establish that the two decisions interact, which is the premise of the present work.

Three studies in Table 2.1 deal with inventory and warehouse decisions using quantum methods. The pilot by Tata Elxsi (2025) ran QAOA directly on IBM quantum hardware to select critical automotive spare parts, but because of the gate fidelity of the device it could select only five parts out of thirty. Ullah et al. (2024) handled warehouse capacity and lead-time limits together in a multi-variable QAOA model and observed that adding soft slack terms for several products makes the number of variables grow quadratically. This observation matters here, because the textbook encoding of the storage-capacity and budget limits also uses slack variables. Nishimura et al. (2025) mapped the positioning of multiple items on warehouse racks to a QUBO and used a hybrid quantum-classical solver, but the circuits stayed at one or two layers because the gates are noisy.

2.5 QUBO Models of Combinatorial Problems

A QUBO problem asks for a binary vector that minimises a quadratic form. Lucas [17] gave Ising and QUBO formulations of many NP-hard problems, including covering, partitioning and scheduling problems, and showed how linear equality constraints become squared penalty terms. Glover, Kochenberger and Du [9] gave a tutorial treatment that includes the conversion of inequality constraints with slack variables and practical rules for choosing penalty weights. Boros and Hammer [4] surveyed pseudo-Boolean optimization, including the reduction of

higher-degree terms to quadratic form with auxiliary variables, which is the technique used in the compact encoding of this project. The common lesson of this literature is that the encoding of constraints, not the objective, decides the size and difficulty of a QUBO.

2.6 The Quantum Approximate Optimization Algorithm

QAOA was introduced by Farhi, Goldstone and Gutmann [7] as a variational algorithm that alternates a cost unitary generated by the problem Hamiltonian with a mixing unitary generated by single-qubit X operators. With p alternating layers the circuit has 2p real parameters, and for $p \to \infty$ it can approximate the adiabatic evolution to the ground state. Hadfield et al. [11] generalised the method to the Quantum Alternating Operator Ansatz, with mixers that preserve problem constraints. Zhou et al. [31] studied the performance and mechanism of QAOA on near-term devices and proposed heuristic parameter initialisation strategies that reduce the cost of the classical optimization loop. Pellow-Jarman et al. [23] examined how the choice of classical optimizer and the ansatz depth affect QAOA on noisy devices, which is directly useful for deciding the circuit depth in this work. QAOA has been applied to vehicle routing by Azad et al. [3] and by Fitzek et al. [8], who showed that a constrained routing problem can be encoded and solved with shallow circuits on small instances.

2.7 Classical Optimization Approaches

The standard exact method for MILP models is branch and bound, introduced by Land and Doig [16], which solves linear relaxations and branches on fractional variables while pruning sub-problems whose bound cannot beat the best known solution. Open-source solvers such as CBC combine branch and bound with cutting planes and heuristics, and the PuLP modelling library [20] makes them accessible from Python. Exact solvers give a proof of optimality, which makes them the natural reference for judging any heuristic. Among heuristics, simulated annealing [15] and greedy constructive methods are commonly used for location problems; both were used in an exploratory scaling study during this project, as reported in Chapter 8.

2.8 Hybrid Quantum-Classical Pipelines

A second group of studies in Table 2.1 places quantum methods inside wider logistics pipelines. Vanderbist et al. (2026) used an alternating hybrid QAOA on simulated maritime shipping networks and found that classical parameter tuning becomes costly once the circuit reaches three layers or more. Afrasiabi et al. (2026) combined QAOA, VQE [24] and quantum annealing on industrial logistics and cross-docking data and reported that the effort

spent on hardware error mitigation grows quickly with problem size. Li (2026) used a hybrid classical-quantum spin Hamiltonian on multichannel warehouse fulfilment data and showed that high penalty factors can give solutions that are not physically feasible. Merali et al. (2025) explored cloud-based hybrid QAOA platforms for supply disruptions in small and medium enterprises, where the communication delay between the local optimizer and the remote quantum back end becomes a bottleneck. The engineering team at Zalando (2025) described how deep demand forecasting could be joined to quantum-informed network paths for SKU-level distribution, but the work stays at the framework stage. Bhattacharyya et al. (2022) mapped multi-node allocation to an Ising Hamiltonian and solved it with QAOA through the Qiskit interface, but performance dropped once the supply nodes passed single digits. On the application side, Raj et al. [27] study dark store portfolios under disruption in quick commerce, and Yang et al. [30] address network planning for micro-fulfilment centres.

2.9 Summary of Limitations and Research Gap

Looking at Table 2.1 as a whole, the limitations fall into a few groups. Penalty handling is the problem in Sinha (2026), Li (2026) and Ullah et al. (2024), where fixed or very large penalty factors lead to local minima or to unusable solutions, and where slack terms add many variables. Circuit depth and the cost of tuning limit Vanderbist et al. (2026) and Nishimura et al. (2025). Hardware noise and error mitigation limit the Tata Elxsi pilot (2025), Vinci et al. (2025) and Afrasiabi et al. (2026). The rest are limited by scope, since they do not cover multi-product inventory, stay at the framework stage or were run only on classical machines.

Several design decisions in this project follow from these limitations. The penalty weight is derived from the data of each instance so that breaking a constraint is never profitable, but it is not made larger than necessary, because an oversized penalty flattens the cost landscape that the optimizer has to search. Inequality constraints are encoded through their minimal violating subsets instead of slack variables, so that the number of qubits does not grow with the size of the capacity and budget values. The delivery-time promise is enforced by not creating variables for links that break it. The circuit depth is kept small and its effect is measured, and a classical exact baseline is run on every instance.

The research gap that remains after this survey is clear. Facility location work does not include product-level stock, inventory work assumes that the store network is already fixed, and very few studies use road-network delivery time as the coverage measure for a hyperlocal setting. A model that treats siting, assignment and product-level stock together, uses real road travel times, is executed with QAOA and is checked against an exact solver and an independent validator is not found in the studies of Table 2.1. This is the combination addressed in this project.

CHAPTER 3
SYSTEM ANALYSIS

System analysis examines how the planning problem is commonly handled today, identifies the weaknesses of that approach, and defines what the proposed system must do and under which conditions it can be built and used. This chapter describes the existing approach, its limitations, the proposed system, the feasibility study, the hardware and software requirements, the functional and non-functional requirements, the constraints and assumptions of the model, and the overall architecture.

3.1 Existing System

Published descriptions of quick-commerce planning, and the classical location-inventory literature, describe a common pattern in which location and inventory are decided by separate processes [30], [27], [6]. The description below summarises this typical practice; it is not a claim about the internal tools of any particular company, which are not public.

3.1.1 Separate store-location and inventory decisions

In the typical sequential approach, a retailer first chooses store sites, then assigns customer areas to the chosen sites, and finally plans the stock of each store, often with a separate tool or team at each step. Each step takes the output of the previous one as fixed. Figure 3.1(a) illustrates this sequence. Because the stock plan never feeds back into the choice of site, a location that looks good on a map can end up with a poor stock mix, and a well stocked store can end up serving the wrong neighbourhood [30].

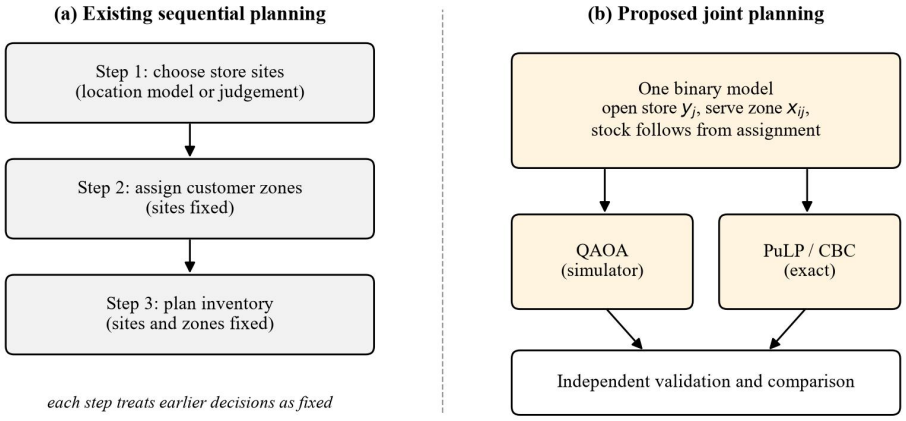


Figure 3.1 Existing sequential planning compared with the proposed joint planning

3.1.2 Conventional optimization approaches

Where optimization is used, the location step is usually solved with a classical facility-location or covering model [22], [5] by a MILP solver or a heuristic, and the inventory step

with a separate replenishment or newsvendor-type model [13]. These models are mature and exact solvers handle them well at moderate sizes, but solving them in sequence does not give the optimum of the combined problem.

3.1.3 Manual or rule-based demand estimation

Demand for a new area is often estimated from rules of thumb such as population, the number of households, or the order history of a similar neighbourhood. When order history is not available, as for a new city or a new operator, the estimate is necessarily coarse, and the quality of every later decision depends on it.

3.1.4 Delivery time, storage capacity and cost

Three operational pressures interact in every plan. The delivery-time promise limits the set of zones each store can serve. The storage capacity of a small dark store limits how much demand it can absorb. The opening, rent and rider costs determine whether serving a zone is worth it at all. A sequential plan typically considers these pressures one at a time.

3.2 Limitations of the Existing System

- **Limited coordination between location and inventory.** Decisions taken in sequence cannot trade a slightly worse site for a much better stock fit, so the combined plan can be far from the joint optimum [29].
- **Difficulty balancing delivery time and cost.** Opening more stores shortens trips but raises fixed cost; without a single objective that prices both, the balance is set by judgement.
- **Storage-capacity and budget constraints.** Capacity is often checked after the assignment is fixed, which can force re-planning or overstocking, and the budget is applied only to the location step.
- **Demand uncertainty and data quality.** Rule-based demand estimates are coarse, and errors propagate through every later step without being visible in the plan.
- **Scaling of optimization models.** The joint problem has a combinatorial number of plans; exact methods are strong at moderate sizes but their run time can grow quickly, and tight budgets make the search harder, as observed in Chapter 8.

Existing quantum studies have their own limitations, as summarised in Section 2.9: they usually address one aspect of the problem at a time, rarely use road-network travel time, and are mostly tested on synthetic data with small instances.

3.3 Proposed System

The proposed system combines store selection, demand-zone assignment and inventory planning in one optimization framework. A single binary model decides which candidate stores to open and which zones each open store serves; the stock of every product at every store then follows exactly from the assignment, so that store choice, service area and inventory are always consistent. The same instance is solved by QAOA and by an exact MILP solver, and both answers are checked by an independent validator. The main elements are the following.

- **OpenStreetMap geographical inputs.** Candidate store sites (48 accepted of 68 fetched supermarket, convenience, general and grocery outlets in the Puducherry mainland bounding box) and 163 demand-zone place nodes are fetched from the Overpass API.
- **OSRM road travel times.** 7,824 store-to-zone legs (48 × 163) are routed over the OpenStreetMap road graph by OSRM; none fell back to a straight-line estimate. A fixed three-minute picking and packing time is added to each leg.
- **Product catalogue inputs.** 25 products are taken from a Blinkit public catalogue sample dated 2026-09-11; unit volume is parsed from the pack size and unit cost is derived from MRP with an assumed 22% retail margin.
- **Population-derived demand.** Census 2011 population figures from Wikidata are spread over 46 demand zones tagged suburb, town or village, and converted to product-level demand with configurable adoption, order-frequency and basket-size coefficients (1,150 demand rows).
- **QUBO formulation.** The constrained model is converted to a QUBO with a compact encoding that penalises minimal violating subsets of the capacity and budget constraints instead of adding slack variables.
- **QAOA with Qiskit.** The QUBO is mapped to an Ising operator and solved with a QAOAAnsatz circuit executed by Qiskit's statevector sampler, with SciPy COBYLA tuning the angles.
- **Classical PuLP/CBC comparison.** The same model with hard constraints is solved exactly by CBC; a proven optimum becomes the reference for the optimality gap.
- **Backend APIs and database integration.** A FastAPI backend exposes REST endpoints for stores, products, demand, data provenance and optimization, and persists every run and its results in PostgreSQL hosted on Supabase.

- **Frontend dashboard.** A React dashboard lets the planner configure budget, delivery limit, stores, products, areas and QAOA settings, preview the qubit count, run both solvers and compare results on a map and in charts, and browse the run history.

Not every element of the original design has been implemented. Table 3.1 separates the implemented features from those that remain proposed, so that the later chapters can be read accurately.

Table 3.1 Implementation status of the main features

Feature	Status	Evidence or remark
Data ingestion from OSM, OSRM, Wikidata and the catalogue	Implemented	backend/ingest.py and services/ingestion; provenance tables
Joint location and assignment model with derived inventory	Implemented	services/data_service.py, qubo_service.py
Compact and slack QUBO encodings	Implemented	services/qubo_service.py
QAOA on statevector simulator	Implemented	services/qaoa_service.py
PuLP/CBC baseline and independent validator	Implemented	classical_service.py, validation_service.py
REST API, PostgreSQL/Supabase persistence, run history	Implemented	routers, models; 60 automated tests
React dashboard with map and comparison	Implemented	frontend/src/pages and components
Live order simulator and fast-path order matching	Proposed (design only)	Shown in Figures 3.3 and 4.4; no endpoint exists
Demand forecasting from order history	Proposed	DEMAND_MODE=historical_orders raises a clear error until order data exist
Product-level stocking as free decision variables	Not adopted	Inventory is derived from the assignment to save qubits
Execution on quantum hardware, noise models	Future work	Only noiseless statevector simulation is used

Table 3.2 summarises the differences between the existing sequential approach and the proposed system.

Table 3.2 Existing system compared with the proposed system

Aspect	Existing (sequential) approach	Proposed system
Decision structure	Location, assignment and stock planned in separate steps	One binary model; stock derived from the assignment
Delivery promise	Checked after siting, often by distance	Road travel time from OSRM; infeasible links never created
Capacity and budget	Checked step by step	Constraints C3 and C4 of the same model
Solution method	Classical models solved in sequence	QAOA (simulator) and exact PuLP/CBC on the identical instance
Validation	Implicit in each tool	Independent validator re-derives every constraint from raw data
Data provenance	Rarely recorded	Every dataset labelled real_public, official, proxy or synthetic
Interface	Separate tools	One web dashboard with map, comparison and run history

3.4 Feasibility Study

3.4.1 Technical feasibility

All components of the system are available as mature open-source software: Python, Qiskit and Qiskit Optimization for the quantum part, SciPy for the classical optimizer, PuLP with its bundled CBC solver for the exact baseline, FastAPI and SQLAlchemy for the backend, PostgreSQL for storage and React for the interface. The quantum circuits are executed on a statevector simulator, which runs on an ordinary laptop for instances of about twenty qubits; the backend enforces this with a configurable limit of 20 qubits. The technical risk lies not in the tools but in the size of the instance that QAOA can be simulated on, which is why the system keeps the classical solver as the reference and reports the qubit count before a run.

3.4.2 Economic feasibility

The software used is open source and free of licence fees, the data sources (OpenStreetMap, the public OSRM demonstration server and Wikidata) are free to use under their licences, and the database is hosted on Supabase. No quantum hardware time was purchased, since all quantum runs are simulated. The project therefore required no direct software or data expenditure beyond the development computer and the hosting plan; the actual hosting tier used should be confirmed by the project team. No claim of financial savings for a quick-commerce operator is made, because the store economics in the model are prototype assumptions.

3.4.3 Operational feasibility

The dashboard presents the decision in the terms a planner uses: stores, areas, budget, delivery limit, cost and service level. A preview endpoint reports the number of qubits before a run, so an oversized instance is rejected with an explanation rather than failing silently. Every number carries its provenance label, so a planner can see which inputs are observed and which are assumptions. Operational use by a real operator would require operator data for store costs, capacities and orders, as noted in Section 3.8.

3.4.4 Schedule feasibility

The work was organised in stages that build on each other: exploratory scripts for the dataset, the classical baseline, the QUBO, QAOA and a scaling study, followed by the backend services, the ingestion pipeline, the database integration and the dashboard. Each stage produced a testable output, which allowed the project to be completed within the mini-project period. The exact calendar of the stages should be confirmed by the project team, as it is not recorded in the repository history.

3.5 Hardware and Software Requirements

The software stack used by the implementation is listed in Table 3.3, with versions taken from the project's requirements and package files. The hardware used for development is listed in Table 3.4; the values that could be measured on the computer used to re-run the experiments for this report are shown, and the other items are marked for confirmation.

Table 3.3 Software requirements of the implemented system

Component	Version used	Role
Operating system	Windows 11 (development and test machine)	Backend, frontend and experiments
Python	3.14 (3.11 or later required)	Optimization, backend and data pipeline
Qiskit / Qiskit Optimization	2.5.2 / 0.7.0	QuadraticProgram, Ising conversion, QAOAAnsatz, StatevectorSampler
qiskit-algorithms	0.4.0 (installed for interoperability only)	Its QAOA class is not used (Section 6.4)
NumPy / SciPy / pandas	2.5.3 / 1.18.1 / 3.0.6	QUBO matrix, COBYLA optimizer, tabular data
PuLP (bundled CBC)	2.9.0	Exact MILP baseline
FastAPI / Uvicorn / Pydantic	0.141.1 / 0.41.0 / 2.13.5	REST API, ASGI server, request validation
SQLAlchemy / psycopg2	2.1.1 / 2.9.12	ORM and PostgreSQL driver
Database	PostgreSQL hosted on Supabase; SQLite for tests	Persistent store for data and runs
Node.js / Vite / React	Node 24, Vite 8.3, React 19.2	Dashboard build and runtime
React Router / axios / React-Leaflet / Recharts / Tailwind CSS	7.18 / 1.20 / 5.0 / 3.10 / 4.3	Routing, HTTP client, maps, charts, styling
pytest / httpx	9.0.1 / 0.28.1	Automated tests

Table 3.4 Hardware used for development and experiments

Component	Value	Remark
Processor	13th Gen Intel Core i5-1335U (10 cores, 12 threads)	Measured on the machine used to re-run the experiments
Memory	16 GB (15.7 GB usable)	Measured; sufficient for statevector simulation up to about 20 qubits
Storage	To be confirmed by the project team	Project and virtual environment need under 2 GB
Network	Internet connection	Needed for Overpass, OSRM, Wikidata and Supabase
Quantum hardware	Not used	All circuits are simulated

The memory requirement deserves a note. A statevector simulator stores 2^n complex amplitudes of 16 bytes each, so 20 qubits need 16 MiB and 30 qubits need 16 GiB. Since every added qubit doubles the memory, the limit is set by the instance size and not by the processor.

3.6 Functional Requirements

The functional requirements describe what the system does. Table 3.5 lists the functions supported by the implementation, with the component responsible for each.

Table 3.5 Functional requirements supported by the implementation

ID	Requirement	Implemented in
FR1	Retrieve and manage candidate stores, demand zones, products and demand (list, filter, create, update, delete; bulk demand upload)	/api/stores, /api/products, /api/demand
FR2	Ingest open data with provenance and report data sources, ingestion runs and status	ingest.py, /api/data
FR3	Store and retrieve road travel times for every store-zone pair	travel_times table, ingestion routing step
FR4	Configure budget, delivery-time limit, stores, products, areas and QAOA settings (depth, iterations, shots, seed) and the CBC time limit	OptimizationForm, RunRequest schema
FR5	Build the optimization instance and preview its size in qubits before solving	/api/optimization/preview, data_service
FR6	Suggest a solvable instance from the stored data	/api/optimization/suggest-instance
FR7	Run QAOA, the classical MILP, or both, on the identical instance	/api/optimization/run, /qaoa, /classical
FR8	Validate every solution against constraints C1 to C5 independently of the solver	validation_service
FR9	Compare objective value, service level, budget, storage, runtime and optimality gap	/api/optimization/{run_id}/comparison
FR10	Persist every run with its configuration and results, and display the run history	optimization_runs and results tables, History page
FR11	Display stores and approximate area locations on a map and show the results in charts	MapView, ComparisonCard, OptimizationResults

3.7 Non-Functional Requirements

The non-functional requirements describe how well the system performs its functions. Table 3.6 lists them with the way each is met in the implementation.

Table 3.6 Non-functional requirements and how they are met

Requirement	Description	How it is met
Reliability	A failure must be reported, not hidden	Failed solvers are saved with status FAILED and a message; unreachable routing is labelled as an estimate; database errors are scrubbed of credentials and explained
Usability	A planner without quantum background can use the system	Inputs in business terms; qubit preview; read-only display of capacity, which is a store property
Maintainability	Parts can be changed independently	Routers contain no optimization logic; services are separate modules; 60 automated tests
Performance	Interactive response for data screens; bounded solve time	CBC time limit (default 60 s); QAOA qubit limit (default 20)
Data integrity	Invalid data must be rejected	Pydantic validation rejects negative costs, demand or budget, zero volume and unknown ids with HTTP 422; bulk demand upload is all-or-nothing

Requirement	Description	How it is met
Security	No secrets in source code	Database URL read from environment; .env excluded from version control. CORS is open and there is no authentication, which must be fixed before deployment
Scalability	Behaviour as instances grow	Classical path handles the stored network; QAOA limited by qubits, reported before solving
Reproducibility	A run can be repeated	Fixed random seeds for angles and sampling; full configuration and solver details stored with each run

3.8 System Constraints and Assumptions

The model and the system operate under the constraints and assumptions listed in Table 3.7. They are stated here because they determine how the results in Chapter 8 may be interpreted.

Table 3.7 Constraints and assumptions of the model and system

Item	Constraint or assumption
Delivery-time limit	A zone can be served by a store only if the travel time plus handling time is within the zone's promise (default 15 minutes); other links are not created
Budget	Total opening cost of the selected stores must not exceed B; values are rounded conservatively (costs up, budget down)
Storage capacity	Total volume of the demand assigned to a store must fit its capacity; volumes rounded up and capacity rounded down
Binary decisions	Opening and assignment variables are binary; each zone is served by at most one store (single sourcing)
Candidate stores and zones	48 candidate outlets and 163 zones from OpenStreetMap; only 46 zones carry demand; outlets are candidate sites, not verified dark stores
Product catalogue	25 products from one catalogue sample; volume and cost are derived estimates
Demand	Population proxy: population × adoption rate (0.06) × orders per user per day (0.35) × items per order (5.0) × category share; not observed orders
Store economics	Opening cost, storage capacity, rider cost (₹3.0 per minute), handling time (3 minutes) and gross margin (18%) are prototype assumptions
Single period	One planning period; no replenishment dynamics or demand variability
QAOA simulator	Noiseless statevector simulation; limited to about 20 qubits on the development machine; no hardware queue or transpilation to a device
Classical baseline	CBC solves the same integerised model; a gap is published only when CBC reports a proven optimum

3.9 System Architecture

The complete data flow of the implemented system is shown in Figure 3.2. Data sources are fetched by the ingestion pipeline, validated and labelled with their provenance, and stored in the Supabase PostgreSQL database. The FastAPI backend reads the database through SQLAlchemy and builds an optimization instance for each request. The instance is passed unchanged to the quantum branch, which builds the QUBO and runs QAOA, and to the classical branch, which builds and solves the MILP with CBC. Both solutions go to the independent validator, which computes the metrics and the optimality gap, and the run and its

results are written back to the database. The React dashboard communicates with the backend only through HTTP and JSON.

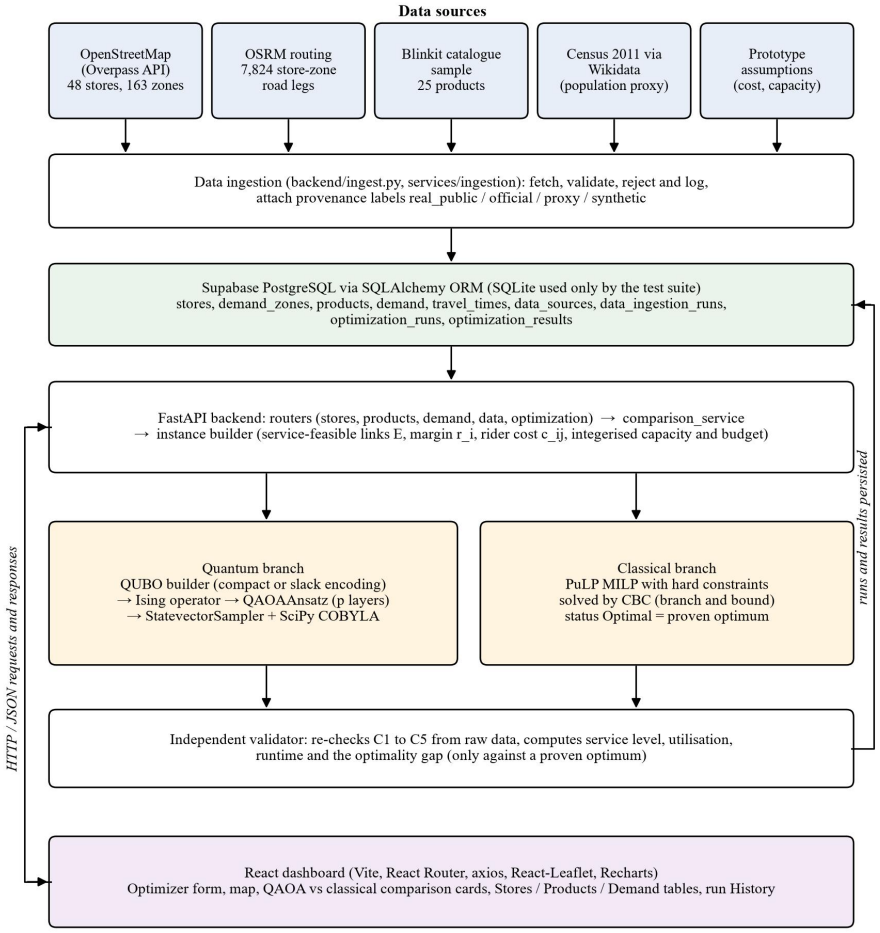


Figure 3.2 Architecture and data flow of the implemented system

The conceptual three-layer architecture of the planning platform, as designed at the start of the project and presented in the conference paper, is shown in Figure 3.3. The user interaction layer, the request handling layer and the data and optimization layer correspond to the dashboard, the FastAPI routers and services, and the database with the solvers in Figure 3.2. The live order simulator and the fast path for order matching in Figure 3.3 belong to the design and have not been implemented, as noted in Table 3.1.

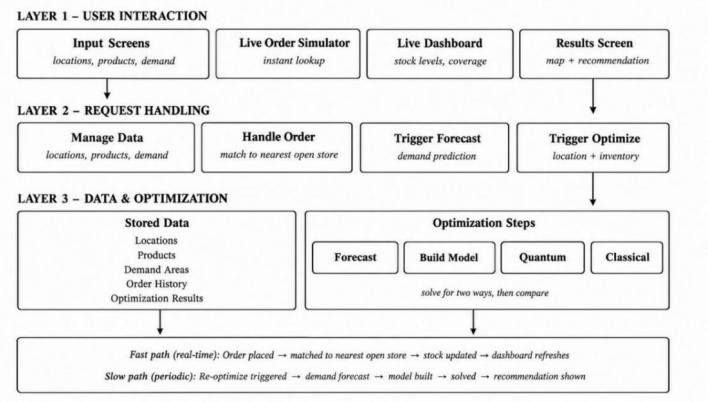


Figure 3.3 Three-layer architecture of the planning platform (design)

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 Design Overview

The proposed framework is organised in four functional modules, as shown in Figure 4.1. Module 1 acquires and pre-processes the data, Module 2 builds the optimization model and its QUBO, Module 3 runs QAOA, and Module 4 decodes and validates the result. A classical MILP baseline runs beside Modules 2 to 4 on the same instance. This modular workflow was first presented in the conference paper and is retained here; the implementation details of each module are given in Chapter 5.

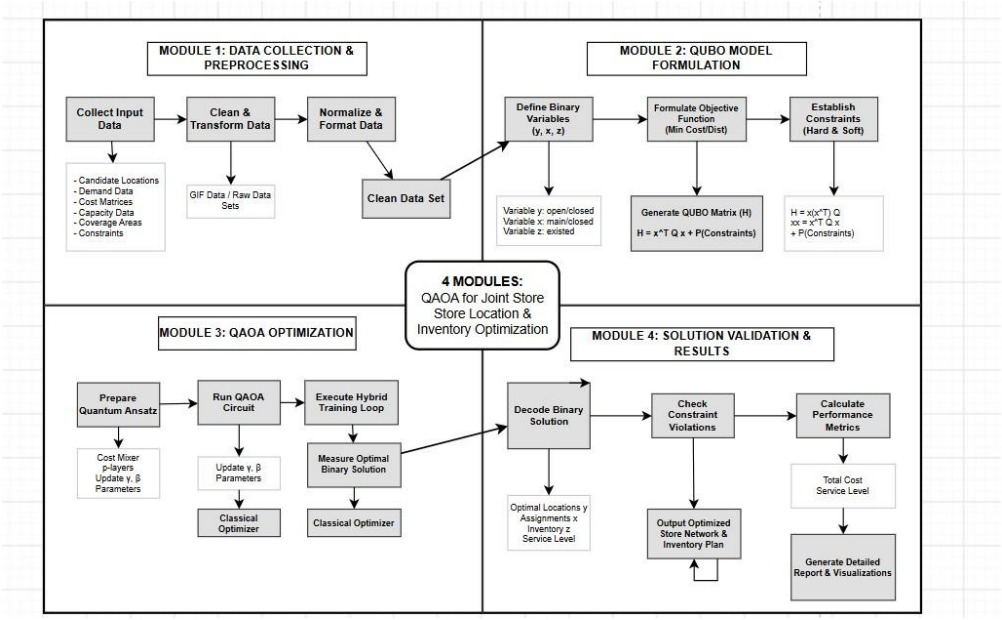


Figure 4.1 Modular workflow of the QAOA framework for joint store location and inventory optimization

Figure 4.1 uses the notation of the conference formulation (variables y, x and z, Section 4.3). In the implemented model the stocking decision z is replaced by inventory derived from the assignment, as explained in Section 4.4; the module structure is otherwise unchanged.

4.2 Datasets and Provenance

The experiments use five datasets, all built from open sources and covering the Puducherry mainland, together with one set of prototype assumptions. Each dataset maps to a parameter of the model, and each carries a provenance label in the database, as listed in Table 4.1.

Table 4.1 Datasets used in the study and their provenance

No.	Dataset	Source	Records	Role in the model	Provenance
1	Candidate store locations	OpenStreetMap (Overpass API), extracted 2026-10-03	48 accepted (68 fetched, 20 rejected)	Candidate sites j	real_public

No.	Dataset	Source	Records	Role in the model	Provenance
2	Demand zones	OpenStreetMap place nodes	163 zones (46 carry demand)	Zones i	real_public
3	Population	Census of India 2011 via Wikidata	7 places; 1 matched to a zone	Input to the demand proxy	official
4	Product catalogue	Blinkit catalogue sample, 2026-09-11	25 products	Products k, volume s_k, cost p_k	real_public (volume, cost estimated)
5	Road travel times	OSRM over OpenStreetMap roads	7,824 legs, 0 fallbacks	Travel time t_ij	real_public
6	Demand	Population proxy model	1,150 rows (46 zones × 25 products)	Demand d_ik	proxy
7	Store economics	Prototype assumptions	48 stores	Opening cost f_j, capacity S_j	synthetic

The number of travel legs follows directly from the other counts, since 48 candidate locations and 163 demand zones give 48 × 163 = 7,824 pairs. Of the 163 OpenStreetMap place nodes, 117 are tagged neighbourhood or quarter and are almost entirely small Auroville settlement plots; spreading the municipal population evenly over all 163 nodes would assign most of the city's demand to these plots. The demand model therefore uses only the 46 zones tagged suburb, town or village. Demand is computed with Equation 4.1.

$$d_{ik} = P_i \times a \times o \times m \times \sigma_k \tag{4.1}$$

Here P_i is the population allocated to zone i, a = 0.06 is the share of residents assumed to use quick commerce, o = 0.35 is the number of orders per adopting resident per day, m = 5.0 is the number of items per order, and σ_k is the share of product k in the category mix of the real catalogue. All coefficients are configurable. The level of demand is therefore an estimate, while its split across products reflects a real catalogue.

4.3 Conceptual Formulation (Conference Paper)

The conference paper states the joint problem as a cost-minimisation model. Let J, I and K be the sets of candidate store locations, customer demand zones and products. The binary variable y_j is 1 when store j is opened, x_{ij} is 1 when zone i is assigned to store j, and z_{jk} is 1 when product k is stocked at store j. The objective is given in Equation 4.2.

$$\min C = \sum_{j \in J} f_j y_j + \alpha \sum_{i \in I} \sum_{j \in J} D_i t_{ij} x_{ij} + \beta \sum_{j \in J} \sum_{k \in K} h_k z_{jk} \tag{4.2}$$

In Equation 4.2, f_j is the opening cost of store j, D_i is the estimated demand of zone i, t_{ij} is the road travel time between store j and zone i, h_k is the inventory holding cost of product k, and α and β are weights that set the relative importance of the delivery cost and the inventory cost against the opening cost. The paper requires each zone to be assigned to exactly one store,

assignment and stocking only at open stores, stocked volume within the storage capacity S_j, total opening cost within the budget B, the delivery-time limit $T_{\max}$, and product availability for every product a zone needs, as in Equations 4.3 to 4.7.

$$\sum_{j \in J} x_{ij} = 1 \quad \forall i \in I \tag{4.3}$$

$$x_{ij} \leq y_j, \quad z_{jk} \leq y_j \quad \forall i \in I, j \in J, k \in K \tag{4.4}$$

$$\sum_{k \in K} s_k z_{jk} \leq S_j y_j \quad \forall j \in J, \qquad \sum_{j \in J} f_j y_j \leq B \tag{4.5}$$

$$x_{ij} = 0 \quad \text{if } t_{ij} > T_{\max} \tag{4.6}$$

$$x_{ij} \leq z_{jk} \quad \text{whenever } d_{ik} > 0 \tag{4.7}$$

This formulation is a useful statement of the problem, but it was refined during implementation for three reasons. First, the stocking variables z_{jk} add one qubit per store-product pair, which for 48 stores and 25 products is 1,200 qubits before any assignment variable. Second, requiring every zone to be served forces the model to serve zones that cannot be reached profitably or within the promise. Third, a cost-only objective has no term that rewards serving demand, so it cannot decide whether a marginal zone is worth serving. The implemented model below addresses all three points, and Table 4.3 summarises the correspondence.

4.4 Implemented Optimization Model

The model implemented in the backend (app/services/data_service.py) uses the same sets and the same notation for opening and assignment, so that the figures of this report remain valid. Its data and derived quantities are defined below.

4.4.1 Sets and parameters

J is the set of candidate stores, I the set of demand zones and K the set of products. For each store j, f_j is the opening cost and S_j the storage capacity in volume units. For each zone i and product k, d_{ik} is the demand quantity, and T_i is the delivery-time promise of zone i in minutes. For each product, s_k is the volume of one unit and p_k its supplier cost. t_{ij} is the road travel time plus handling time from store j to zone i, B is the opening budget, g is the gross margin per rupee of demand value served, ρ is the rider cost per minute and m is the average number of items per order. The derived quantities per zone are given in Equations 4.8 to 4.10.

$$v_i = \sum_{k \in K} d_{ik} s_k , \qquad q_i = \sum_{k \in K} d_{ik} \tag{4.8}$$

$$r_i = g \sum_{k \in K} d_{ik} p_k \tag{4.9}$$

$$c_{ij} = \rho \, t_{ij} \, n_i, \qquad n_i = \lceil q_i / m \rceil \tag{4.10}$$

In these equations v_i is the storage volume needed to serve zone i, q_i its demand in units, r_i the margin earned by serving it, n_i the number of orders per period and c_{ij} the rider cost of serving zone i from store j. The default values are g = 0.18, ρ = ₹3.0 per minute and m = 5.0.

4.4.2 Service-feasible links

The delivery-time constraint is handled structurally. Only the store-zone pairs that meet the promise are created as variables, as defined by Equation 4.11.

$$E = \{(i, j) \in I \times J : t_{ij} \leq T_i\} \tag{4.11}$$

A link outside E is not a poor option but no option, and leaving it out removes the corresponding qubit entirely. This is the largest single saving in the formulation.

4.4.3 Decision variables

The decision variables are binary, as shown in Equation 4.12, and are summarised with all symbols in Table 4.2.

$$y_j \in \{0,1\} \quad \forall j \in J, \qquad x_{ij} \in \{0,1\} \quad \forall (i, j) \in E \tag{4.12}$$

Table 4.2 Variables and parameters of the implemented model

Symbol	Meaning	Unit or domain
y_j	1 if candidate store j is opened	binary
x_{ij}	1 if zone i is served by store j, defined only for (i, j) in E	binary
I_{jk}	inventory of product k required at store j (derived, Eq. 4.18)	units
f_j, S_j	opening cost and storage capacity of store j	₹, volume
d_{ik}	demand for product k in zone i	units per period
s_k, p_k	unit volume and unit supplier cost of product k	volume, ₹
t_{ij}, T_i	travel time from store j to zone i, and promise of zone i	minutes
v_i, r_i, c_{ij}	volume, margin and rider cost derived in Eqs. 4.8 to 4.10	volume, ₹, ₹
B, g, ρ, m	budget, gross margin, rider cost per minute, items per order	₹, ratio, ₹/min, items
λ	penalty weight of the QUBO	₹

4.4.4 Objective function

The objective is to maximise the net operating margin, which is the margin on the demand actually served minus the rider cost and the opening cost, as in Equation 4.13.

$$\max Z = \sum_{(i,j)\in E} (r_i - c_{ij}) x_{ij} - \sum_{j\in J} f_j y_j \tag{4.13}$$

Written as a minimisation, which is the form used by the QUBO, the objective becomes $C = \sum_j f_j y_j + \sum c_{ij} x_{ij} - \sum r_i x_{ij}$, which equals −Z. The first two terms correspond to the opening and delivery terms of Equation 4.2, with $\alpha D_i t_{ij}$ replaced by the rider cost $\rho n_i t_{ij}$. The third term rewards served demand and is needed because a zone may be left unserved. The holding-cost term of Equation 4.2 does not appear because inventory is not a free variable. All objective values reported in this report, including ₹14,716.14, are values of Z, the net margin in rupees, not total cost.

4.4.5 Constraints

The model has five constraints. Constraint C1 in Equation 4.14 allows each zone to be served by at most one store. It is an inequality on purpose: leaving a zone unserved is allowed and simply forfeits its margin, so the model decides whether serving a zone is worth it.

$$\sum_{j:(i,j)\in E} x_{ij} \le 1 \quad \forall i \in I \tag{4.14}$$

Constraint C2 in Equation 4.15 couples assignment to opening: a zone can be served, and stock can be placed, only at a store that is opened. This is the constraint that joins the location and inventory halves of the problem.

$$x_{ij} \le y_j \quad \forall (i,j) \in E \tag{4.15}$$

Constraint C3 in Equation 4.16 limits the volume assigned to each store to its capacity, and constraint C4 in Equation 4.17 limits the total opening cost to the budget. Both use integer coefficients rounded in the conservative direction: required volume and opening cost are rounded up, capacity and budget are rounded down, so that any plan feasible in the rounded model is feasible in the original one.

$$\sum_{i:(i,j)\in E} \lceil v_i \rceil x_{ij} \le \lfloor S_j \rfloor y_j \quad \forall j \in J \tag{4.16}$$

$$\sum_{j\in J} \lceil f_j \rceil y_j \le \lfloor B \rfloor \tag{4.17}$$

Constraint C5, the delivery-time promise, is satisfied by construction because only links in E exist. Product availability, which required Equation 4.7 in the conceptual model, is also satisfied by construction, because the inventory of each store is derived from the demand of the zones it serves, as in Equation 4.18.

$$I_{jk} = \sum_{i:(i,j)\in E} d_{ik} x_{ij} \quad \forall j \in J, k \in K \tag{4.18}$$

Table 4.3 Conceptual formulation compared with the implemented model

Element	Conference formulation	Implemented model
Objective	Minimise opening + α·delivery + β·holding cost	Maximise margin served − rider cost − opening cost
Stocking	Binary z_{jk} per store and product	Derived inventory I_{jk} (Eq. 4.18); no qubits
Assignment	Exactly one store per zone	At most one store per zone (C1)
Coupling	$x_{ij} \le y_j, z_{jk} \le y_j$	$x_{ij} \le y_j$ (C2)
Capacity and budget	Inequalities with slack variables	Inequalities C3, C4; compact or slack encoding
Delivery limit	Penalty on pairs above $T_{\max}$	Structural: links outside E are not created (C5)
Product availability	$x_{ij} \le z_{jk}$ when $d_{ik} > 0$	Implied by derived inventory

4.5 QUBO Formulation

4.5.1 General form

QAOA can minimise only an unconstrained quadratic function of binary variables, so the constraints are moved into the objective as penalty terms. The general QUBO has the form of Equation 4.19, where b collects the decision variables and any auxiliary variables, C(b) is the operational objective −Z, and $P_r(b)$ is the penalty for the r-th constraint group, which is zero when the constraint holds and positive when it is broken.

$$\min_{\mathbf{b}\in\{0,1\}^n} H(\mathbf{b}) = C(\mathbf{b}) + \sum_{r=1}^{m} P_r(\mathbf{b}) = \mathbf{b}^T Q \mathbf{b} + \text{const} \tag{4.19}$$

4.5.2 Compact encoding (default)

The implementation uses a compact encoding by default. Constraint C1 is enforced by a pairwise penalty on every two stores that could serve the same zone, Equation 4.20, which is zero when at most one of them serves it. Constraint C2 is enforced by Equation 4.21, which is positive exactly when a zone is served from a closed store.

$$P_1 = \lambda \sum_{i\in I} \sum_{j<j'} x_{ij} x_{ij'} \tag{4.20}$$

$$P_2 = \lambda \sum_{(i,j)\in E} x_{ij}(1 - y_j) \tag{4.21}$$

The inequalities C3 and C4 are not converted with slack variables. Instead, the implementation enumerates their minimal violating subsets. For store j, a set M of zones is a violating subset if $\sum_{i\in M} \lceil v_i \rceil > \lfloor S_j \rfloor$, and it is minimal if no proper subset of M is already violating. Writing $\boldsymbol{\mathcal{M}}_j$ for the family of minimal violating subsets of store j and $\boldsymbol{\mathcal{M}}_B$ for that of the budget, the penalties are those of Equations 4.22 and 4.23.

$$P_3 = \lambda \sum_{j\in J} \sum_{M\in \boldsymbol{\mathcal{M}}_j} \prod_{i\in M} x_{ij} \tag{4.22}$$

$$P_4 = \lambda \sum_{M\in \boldsymbol{\mathcal{M}}_B} \prod_{j\in M} y_j \tag{4.23}$$

A product of binary variables is 1 only when every variable in it is 1, so each term is positive exactly when all the zones (or stores) of a violating set are selected together. A constraint that no reachable plan can violate produces no terms at all. A product of two variables is already quadratic. A product of k ≥ 3 variables is reduced to quadratic form with k − 2 auxiliary variables using the Rosenberg substitution [4]: an auxiliary variable w is forced to equal the product ab by the penalty of Equation 4.24, which is zero exactly when w = ab and positive otherwise.

$$P_{\mathrm{AND}}(w, a, b) = \lambda(3w + ab - 2wa - 2wb) \tag{4.24}$$

4.5.3 Penalty weight

A penalty must exceed the largest gain that breaking the constraint could bring, or cheating becomes profitable and the QUBO minimum is not a valid plan. The implementation derives the weight from each instance with Equation 4.25.

$$\lambda = 2\left(\max_i r_i + \max_j f_j + \max_{(i,j)} c_{ij}\right) + 1 \tag{4.25}$$

The weight is not made larger than necessary, because an oversized penalty compresses the real differences between plans into a small fraction of the coefficient range and flattens the landscape that QAOA has to search, which is the sensitivity to penalty factors reported in the literature (Section 2.9). For the prototype instance Equation 4.25 gives λ = 92,526.42.

4.5.4 Slack encoding (alternative)

The textbook alternative, also implemented, converts each inequality $\sum_l a_l b_l \leq R$ into an equality with a binary-encoded slack and squares it [9], as in Equation 4.26.

$$P = \lambda\left(\sum_{l} a_l b_l + \sum_{q=0}^{n_s-1} 2^q u_q - R\right)^2, \qquad n_s = \lceil \log_2(R+1) \rceil \tag{4.26}$$

The slack bits u_q carry no business meaning and are discarded when the solution is decoded. Their number grows with the logarithm of each right-hand side and they are needed for every capacity constraint and the budget. On the prototype instance the slack encoding needs 62 qubits (10 decision and 52 slack), while the compact encoding needs 11 (10 decision and 1 auxiliary); on the 4-store OpenStreetMap instance the figures are 79 and 15. Both encodings describe the same feasible set and differ only in how infeasible points are priced.

4.5.5 Number of qubits

One qubit is needed for every binary variable, so the register size is given by Equation 4.27, where n_{aux} is the number of AND-gadget variables (compact) or slack bits (slack).

$$n = |J| + |E| + n_{\text{aux}} \tag{4.27}$$

If every zone could reach every store, $|E|$ would be $|I| \times |J|$; for the full set of 163 zones and 48 stores the decision variables alone would then be 48 + 7,824 = 7,872. With the 46 demand zones and the 15-minute promise, the stored network has 1,056 feasible links, so the decision variables number 1,104. Both figures are far beyond what a statevector simulator can hold, as discussed in Chapter 8.

4.6 Worked Example on the Prototype Instance

The prototype instance used for development and testing has three candidate stores, three areas and three products, with the demo values seeded in the database. These values are illustrative assumptions and are not observed operator data; the travel times of this instance are straight-line estimates because the demo areas have no routed legs. Table 4.4 lists the derived data.

Table 4.4 Derived data of the prototype instance (budget B = ₹20,000, promise 15 minutes)

Item	Lawspet	Marie Oulgaret	Venkata Nagar
Demand units q_i (milk, bread, water)	2,625	1,925	2,250
Volume v_i	2,275.0	1,687.5	1,950.0
Margin r_i (₹)	19,719.00	14,517.00	16,902.00
Orders n_i	525	385	450
Rider cost from Venkata Nagar Hub 1 (₹)	17,343.71	10,646.86	4,050.00
Rider cost from Lawspet Hub (₹)	4,725.00	not in E	14,866.04
Rider cost from Marie Oulgaret Hub (₹)	not in E	3,465.00	12,444.38

The stores have opening costs of ₹7,800 (Venkata Nagar Hub 1), ₹9,200 (Lawspet Hub) and ₹8,100 (Marie Oulgaret Hub), and capacities of 3,800, 4,200 and 3,600 volume units. Two of the nine store-area pairs exceed the promise, so E has seven links and the model has 3 + 7 = 10 decision variables. Opening all three stores would cost ₹25,100, above the budget, so {all three stores} is the single minimal violating subset of the budget. It has three members and therefore needs one AND-gadget variable, which gives 11 qubits in total.

The optimum opens Venkata Nagar Hub 1 and Lawspet Hub (₹17,000), serves Venkata Nagar and Marie Oulgaret from Venkata Nagar Hub 1 and Lawspet from Lawspet Hub. Its margin is 19,719 + 14,517 + 16,902 = ₹51,138, its rider cost 4,050 + 10,646.86 + 4,725 = ₹19,421.86, and therefore Z = 51,138 − 19,421.86 − 17,000 = ₹14,716.14. Venkata Nagar Hub 1 carries 1,950 + 1,687.5 = 3,637.5 volume units of its 3,800 capacity, so C3 is satisfied. This value was confirmed by enumerating all 2^{11} = 2,048 bit strings of the QUBO, by CBC and by QAOA, as reported in Chapter 8.

4.7 Ising Mapping and the QAOA Procedure

4.7.1 From QUBO to Ising Hamiltonian

The QUBO is converted into an Ising Hamiltonian by replacing every binary variable with a spin operator through Equation 4.28, where Z_l is the Pauli-Z operator of qubit l, whose eigenvalues +1 and −1 correspond to b_l = 0 and 1.

$$b_l = \frac{1 - Z_l}{2} \tag{4.28}$$

Substituting into $\mathbf{b}^T Q \mathbf{b}$ gives the cost Hamiltonian of Equation 4.29. Each quadratic coefficient of the QUBO becomes one two-qubit ZZ interaction and each linear coefficient a single-qubit term; for example $b_1 b_2 = (1 - Z_1 - Z_2 + Z_1 Z_2)/4$.

$$H_C = \sum_l h_l Z_l + \sum_{l<l'} J_{ll'} Z_l Z_{l'} + \text{const} \tag{4.29}$$

The raw coefficients of H_C reach about ₹91,500 on the prototype, so even a small angle γ would rotate phases through many full turns and make the angle landscape impossible to search. The implementation therefore divides the operator by its largest coefficient and drops the identity term, Equation 4.30. Division by a positive constant preserves the ranking of all bit strings and is equivalent to rescaling γ; all reported rupee values are computed from the original unscaled QUBO.

$$\widetilde{H}_C = \frac{H_C - \text{const}}{\max |c|} \tag{4.30}$$

4.7.2 Cost and mixer operators and the parameterized circuit

QAOA prepares the uniform superposition $|+\rangle^{\otimes n}$ and applies p alternating layers of the cost unitary $e^{-i\gamma_l \tilde{H}_C}$ and the mixer unitary $e^{-i\beta_l H_M}$, where the mixer is the transverse-field operator of Equation 4.31. The resulting state is given by Equation 4.32.

$$H_M = \sum_{l=1}^{n} X_l \tag{4.31}$$

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_M} e^{-i\gamma_l \tilde{H}_C} |+\rangle^{\otimes n} \tag{4.32}$$

The circuit has 2p real parameters. It is constructed with Qiskit's QAOAAnsatz class and measured in the computational basis. For p = 3 the prototype circuit has depth 71 and the 15-qubit OpenStreetMap circuit has depth 86.

4.7.3 Classical parameter optimization

A classical optimizer tunes the angles to minimise the expected cost. The expectation is estimated from S measured bit strings $\mathbf{b}^{(s)}$ by evaluating the original QUBO on each sample, Equation 4.33.

$$F(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \psi | H | \psi \rangle \approx \frac{1}{S} \sum_{s=1}^{S} H\left(\mathbf{b}^{(s)}\right) \tag{4.33}$$

The optimizer is SciPy's COBYLA [25], a derivative-free method suited to noisy objective estimates, started from angles drawn uniformly from [0, π] with a fixed seed and an initial trust-region radius of 0.5. It stops when the trust region reaches its lower bound or when the iteration limit is reached. The circuit is executed by Qiskit's StatevectorSampler, which computes the exact state and draws the requested number of shots from it; no noise model is applied and no quantum hardware is used.

4.7.4 Measurement, decoding and feasibility validation

After the angles are optimised, the circuit is sampled once more with four times the number of shots. Every distinct bit string is decoded through the variable index built with the QUBO: the bits of y_j give the opened stores, the bits of x_{ij} the assignments, and auxiliary or slack bits are dropped. Each decoded plan is passed to the independent validator, which recomputes C1 to C5 from the raw instance data without using the QUBO, the penalty weight or the solver's own status. The returned solution is the feasible sample with the highest net margin; if no

sample is feasible, the lowest-cost sample is returned and explicitly marked infeasible. The complete procedure is shown in Figure 4.2.

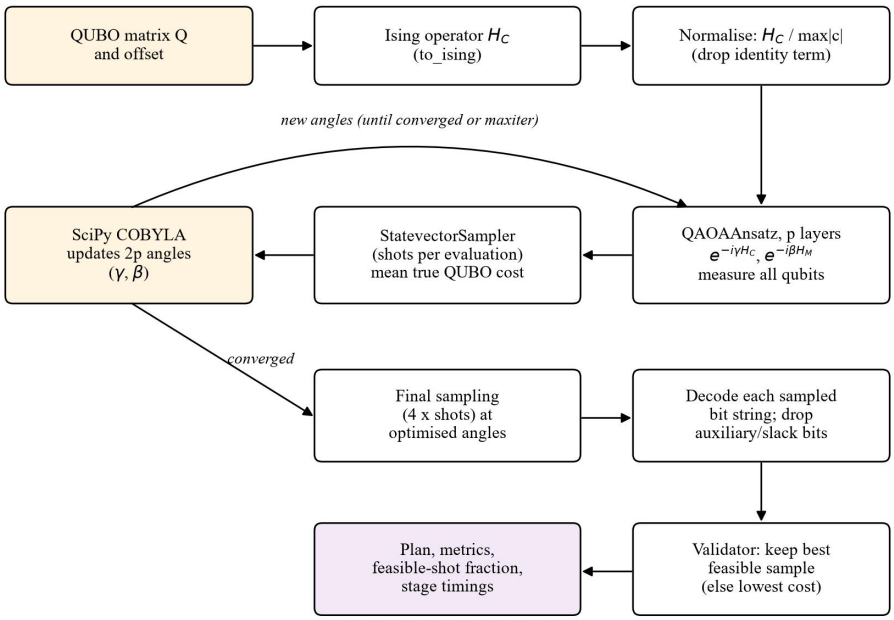


Figure 4.2 QAOA workflow as implemented in the backend

4.8 Classical Baseline

The MILP baseline uses the same instance and the same integerised constraint data, and states C1 to C4 as hard linear constraints with the objective of Equation 4.13. It is modelled in PuLP and solved by CBC with branch and bound [16] under a time limit (60 s by default). When CBC reports the status Optimal within the limit, its value is a proven optimum for the model and becomes the reference for the optimality gap of both solvers. When the limit is reached, the value is only the best solution found and no gap should be published.

4.9 Evaluation Measures

Both solutions are scored by the same validator code, so any difference in the numbers comes from the solutions and not from the measurement. The service level, storage utilisation and budget utilisation are defined in Equations 4.34 to 4.36, where $J^{\cdot}$ is the set of opened stores.

$$\text{Service level} = \frac{\sum_{i \text{ served}} q_i}{\sum_{i \in I} q_i} \tag{4.34}$$

$$\text{Storage utilisation} = \frac{\sum_{(i,j)} v_i x_{ij}}{\sum_{j \in J^{\cdot}} S_j} \tag{4.35}$$

$$\text{Budget utilisation} = \frac{\sum_{j \in J^{\cdot}} f_j}{B} \tag{4.36}$$

For the maximisation objective, the optimality gap of a solver with value Z against a proven optimum $Z^{\cdot}$ is given by Equation 4.37; a positive gap means that the solver fell short of the optimum. The number of constraint violations and the wall-clock runtime are also recorded.

$$\text{Gap} = \frac{Z^{\cdot} - Z}{|Z^{\cdot}|} \times 100\% \tag{4.37}$$

4.10 Flow Charts

The design of the complete project is shown as a flow chart in Figure 4.3. It follows the data from the first collection step to the final plan displayed on the map, and it shows the decisions that control the QAOA loop and the feasibility check. The classical MILP branch runs next to the quantum branch on the same input and joins it at the comparison step. In the implementation, the penalty weight is derived from the instance by Equation 4.25 rather than retuned in a loop; an infeasible QAOA result is reported as such, and automatic retuning remains a design option.

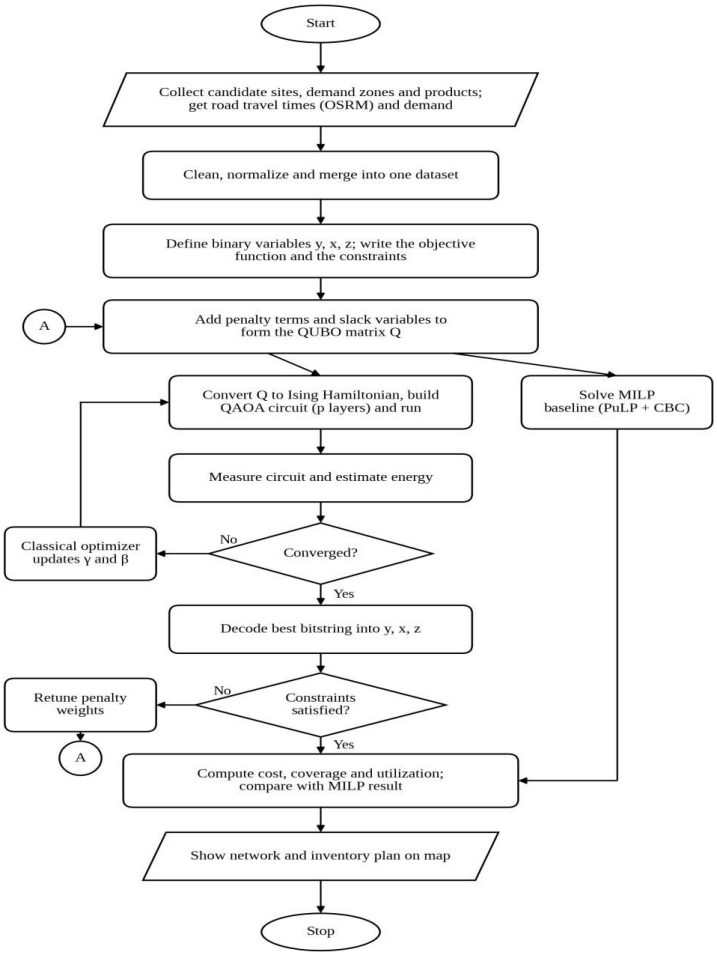


Figure 4.3 Flow chart of the QAOA-based framework for joint store location and inventory optimization

The fast path of the planner, shown in Figure 4.4, was designed to match a live order to the nearest open store that has the item in stock, using only the stored data and no optimization.

The slow path, shown in Figure 4.5, starts when the planner requests a re-optimization: the latest data are loaded, the QUBO and MILP models are built and solved, each solution is validated and the comparison is shown and stored. The slow path corresponds to the implemented optimization endpoints, with the population-based estimate in place of a forecast; the fast path is a design that has not been implemented.

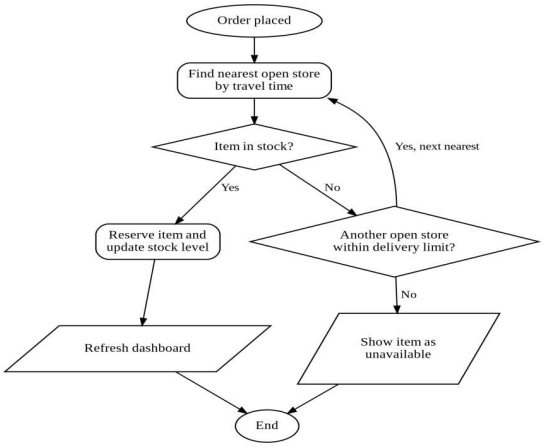


Figure 4.4 Flow chart of the fast path for live order handling (design, not implemented)

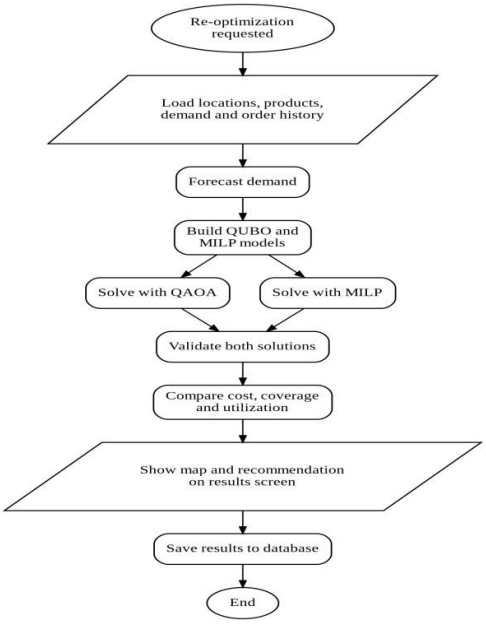


Figure 4.5 Flow chart of the slow path for re-optimization

CHAPTER 5
MODULES

The system is divided into six modules. The first four follow the modular workflow of Figure 4.1, namely data collection and preprocessing, model and QUBO formulation, QAOA optimization, and solution validation. The fifth module is the classical MILP baseline and the sixth is the planner interface that connects the planner to the other modules. The block diagrams in this chapter were drawn during the design stage with the conference notation; the text of each section states how the implementation realises the module.

5.1 Module 1: Data Collection and Preprocessing

Module 1 collects and prepares the data, as shown in Figure 5.1. The candidate store sites and the demand zones are fetched from OpenStreetMap through the Overpass API, the products are taken from the Blinkit catalogue sample, the road travel times are obtained from OSRM, and the population is retrieved from Wikidata. The ingestion manager runs these steps in order, validates every record, logs each rejected record with its reason, and writes the accepted rows to the database with a provenance label.

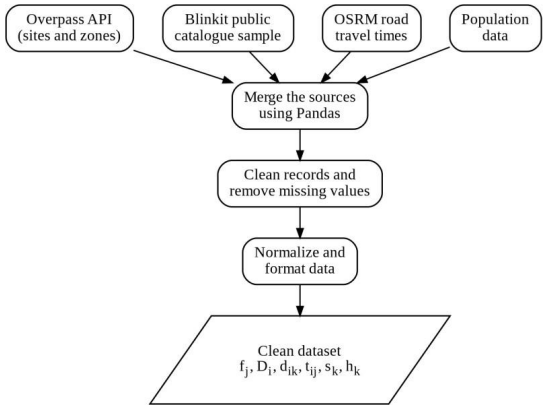


Figure 5.1 Block diagram of Module 1 (data collection and preprocessing)

Several cleaning rules are applied. Outlets outside the Puducherry mainland bounding box (11.87–12.06 N, 79.74–79.88 E) are rejected, because an area query on "Puducherry" also returns the exclaves of Mahé and Yanam, hundreds of kilometres away, which would corrupt every distance. Pack-size strings such as "500 g" or "750 ml" are parsed into a litre-equivalent volume. Any routing leg that OSRM cannot serve is stored as a labelled straight-line estimate, so that an estimate is never mistaken for a route. The output is a database holding 48 candidate locations, 163 demand zones, 25 products, 1,150 demand rows and 7,824 travel legs.

5.2 Module 2: Model and QUBO Formulation

Module 2 builds the optimization instance and its QUBO, as shown in Figure 5.2. The instance builder selects the requested stores, areas and products, computes the derived volume, margin, orders and rider cost of each zone, creates only the service-feasible links, and integerises the capacity and budget data in the conservative direction. The QUBO builder then writes the negated objective, the single-sourcing and coupling penalties, and the capacity and budget penalties through minimal violating subsets with AND gadgets, or, if the slack encoding is chosen, through binary slack variables. The output is the QUBO matrix Q with its variable index, the penalty weight and a summary that reports the number of qubits.

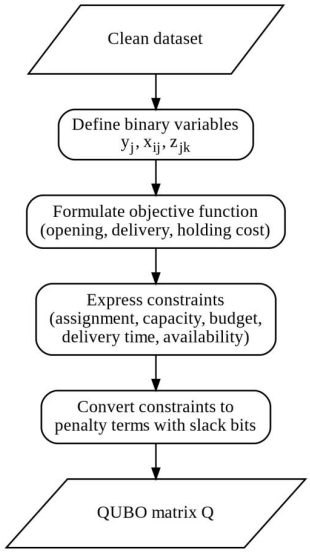


Figure 5.2 Block diagram of Module 2 (QUBO model formulation)

5.3 Module 3: QAOA Optimization

Module 3 is the quantum part of the system; its steps follow the QAOA workflow of Figure 4.2. The QUBO is converted into an Ising operator, normalised, and used to build a QAOAAnsatz circuit with p layers. The circuit starts from an equal superposition of all bit strings; each layer applies the cost unitary with angle γ and the mixer with angle β. The circuit is sampled by the statevector sampler, the mean cost of the samples is passed to COBYLA, and COBYLA proposes new angles until it converges or reaches its iteration limit. The module refuses instances above the configured qubit limit (20 by default) with a message that states the qubit count.

In the experiments the circuit depth was set to three, with 150 iterations and 2,048 shots per evaluation unless stated otherwise, and the circuits were simulated without hardware noise.

5.4 Module 4: Solution Validation and Results

Module 4 checks each answer and turns it into a plan, as shown in Figure 5.3. The bit string is decoded into the opened stores and the store assigned to each zone, and the validator checks C1 (at most one store per zone), C2 (no zone served from a closed store), C3 (capacity), C4 (budget) and C5 (delivery time, using the measured travel times of the instance). It then computes the net margin, revenue, opening and rider cost, service level, storage and budget utilisation, and a list of any violations. The validator never reads the QUBO, the penalty weight or the solver status, which is what makes a reported violation meaningful. In the design of Figure 5.3, an infeasible result triggers an adjustment of the penalty weights; in the implementation the infeasible result is reported and the weight is derived from the instance.

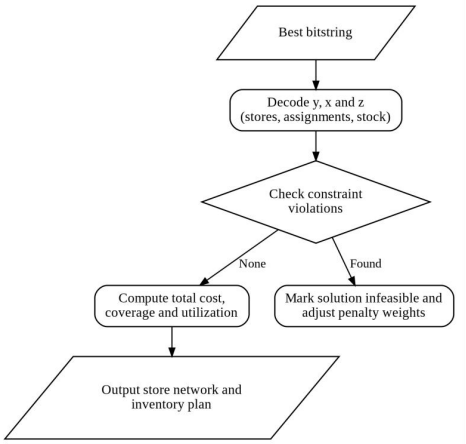


Figure 5.3 Block diagram of Module 4 (solution validation and results)

5.5 Module 5: Classical MILP Baseline

Module 5 is the classical baseline, shown in Figure 5.4. It receives the same instance and builds a PuLP model with the same objective and constraints, written directly as linear constraints with no penalty terms or slack variables. The model is solved by CBC, and the module reports the PuLP status, whether optimality was proven, the objective, the plan and the runtime. A proven optimum is the reference for the optimality gap.

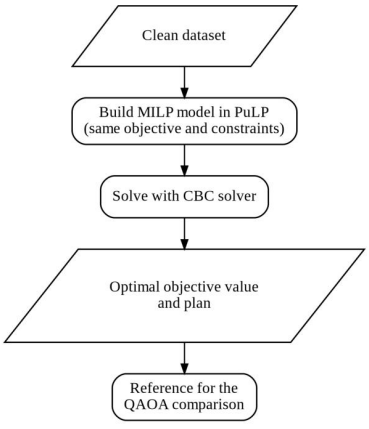


Figure 5.4 Block diagram of Module 5 (classical MILP baseline)

5.6 Module 6: Planner Interface

Module 6 is the planner interface, whose two designed paths are shown in Figure 5.5. The implemented interface realises the slow path: the planner chooses the budget, delivery limit, stores, products and areas, previews the qubit count, runs QAOA and the classical solver, and sees the selected stores on a map with the comparison of both solvers. The fast path for live order matching is part of the design and is not implemented. The interface is built with React and communicates with the FastAPI backend, which stores every run in the database through SQLAlchemy.

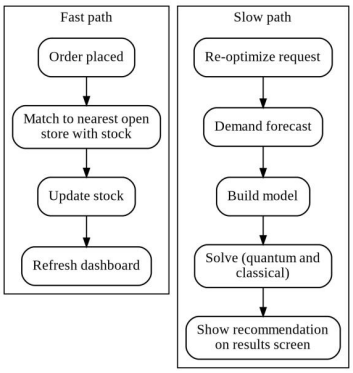


Figure 5.5 Block diagram of Module 6 (planner interface with fast and slow paths)

The inputs, tools and outputs of the six modules are collected in Table 5.1.

Table 5.1 Inputs, tools and outputs of the modules

Module	Input	Main tools	Output
1 Data collection and preprocessing	Overpass, OSRM, Wikidata, catalogue file	Requests, pandas, SQLAlchemy	Database tables with provenance
2 Model and QUBO formulation	Instance request, database rows	Python, NumPy, Qiskit Optimization	Instance, QUBO matrix Q, qubit count
3 QAOA optimization	QUBO and settings p, iterations, shots, seed	Qiskit QAOAAnsatz, StatevectorSampler, SciPy COBYLA	Sampled bit strings and best feasible plan
4 Solution validation	Decoded plan, raw instance	Python	Metrics, violations, comparison
5 MILP baseline	Instance	PuLP, CBC	Proven optimum or best found plan
6 Planner interface	Planner inputs, API responses	React, Vite, React-Leaflet, Recharts, FastAPI	Map, comparison cards, run history

CHAPTER 6
IMPLEMENTATION

This chapter describes the implemented software. The backend is a Python package under backend/app, the data pipeline under backend/ingest.py and app/services/ingestion, and the dashboard under frontend/src. Earlier exploratory scripts (stage1_dataset.py to stage7_pondicherry.py) are referred to only in Section 8.8. The code listings below are short excerpts from the actual source files. In the code, the opening variable is named x_i and the assignment variable $s_{i,a}$ (store i serves area a); they correspond to y_j and x_{ij} in this report.

6.1 Backend Structure

The backend follows a strict dependency direction: routers handle HTTP only and call the comparison service, which calls the data, QUBO, QAOA, classical and validation services. Table 6.1 lists the main files.

Table 6.1 Main backend files and their responsibilities

File	Responsibility
app/main.py	FastAPI application, CORS, error handlers, /health and documentation routes
app/config.py	Settings from environment variables (database URL, margin, rider cost, QAOA and CBC limits)
app/database.py	SQLAlchemy engine and sessions, connection check with credential scrubbing and connect timeout
app/models/	ORM models: stores, products, demand, demand zones, travel times, provenance, runs and results
app/schemas/	Pydantic request and response models with validation
app/routers/	stores, products, demand, data and optimization endpoints
app/services/data_service.py	Seed loading and the shared instance builder (full model statement)
app/services/qubo_service.py	Constrained reference model and the compact and slack QUBO encodings
app/services/qaoa_service.py	QAOA loop on Qiskit primitives; brute-force check for small QUBOs
app/services/classical_service.py	PuLP model and CBC solve
app/services/validation_service.py	Independent constraint checking and metrics
app/services/comparison_service.py	Orchestration, persistence and comparison with optimality gap
app/services/ingestion/	Store, zone, product, demand and routing ingestion with logging

6.2 Database Integration

The data model is defined with SQLAlchemy 2 and created automatically at start-up. The intended target is PostgreSQL hosted on Supabase; the connection string is read from the DATABASE_URL environment variable and is never written in the source. The engine uses pre-ping and connection recycling, because a pooler may close idle connections, and a 15-second connect timeout so that an unreachable host fails quickly with a useful message. Error

messages are scrubbed of any user name and password before they are logged or returned. The main tables and their relationships are shown in Figure 6.1.

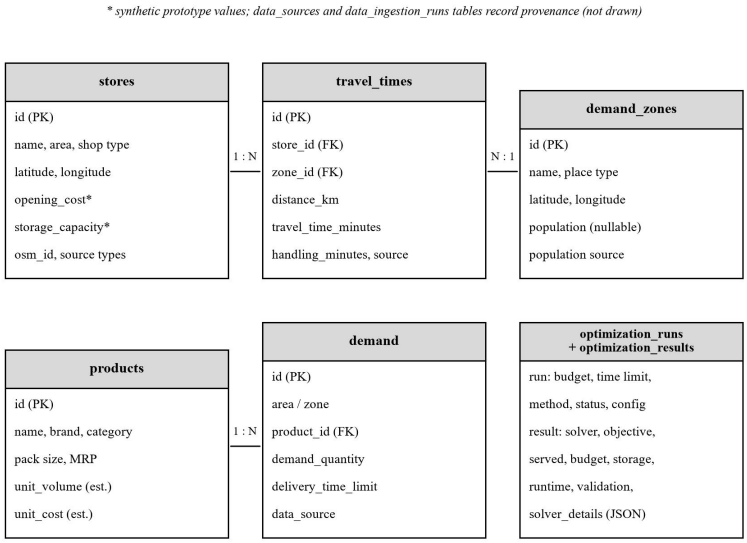


Figure 6.1 Main database tables and relationships

6.3 REST API

The API is documented automatically by FastAPI at /docs. Its endpoints are listed in Table 6.2.

Table 6.2 REST API endpoints of the backend

Method	Path	Purpose
GET	/, /health	Service metadata; liveness and database check
GET, POST, PUT, DELETE	/api/stores, /api/stores/{id}	List, filter, create, update and delete stores
GET, POST, PUT, DELETE	/api/products, /api/products/{id}	Manage products (delete cascades demand rows)
GET, POST, PUT, DELETE	/api/demand, /api/demand/bulk	Manage demand; bulk insert is all-or-nothing
GET, POST	/api/data/sources, /status, /ingestion-runs, /refresh	Provenance, status and manual data refresh
POST	/api/optimization/preview	Size an instance (qubits) without solving
GET	/api/optimization/suggest-instance	Suggest a solvable instance from stored data
POST	/api/optimization/run, /qaoa, /classical	Run both solvers or one of them
GET	/api/optimization/runs, /{run_id}, /{run_id}/comparison	Run history, stored results and comparison

Invalid inputs are rejected with HTTP 422 and a message that names the field: negative demand, cost or budget, a zero unit volume, an unknown store or product id, or a budget below the cheapest store. An instance above the qubit limit returns an error with the qubit

count, and a run in which every requested solver failed is stored with status FAILED and returns the solver name and error.

6.4 QAOA Implementation

The QAOA solver is written as an explicit loop over Qiskit primitives rather than with the QAOA class of qiskit-algorithms. The project measured that class first: with qiskit-algorithms 0.4.0 on Qiskit 2.5, its run time on trivial diagonal operators rose from 0.18 s at 3 qubits to 12.26 s at 9 qubits and did not finish within 200 s at 11 qubits, roughly twenty times per two qubits, whereas statevector growth predicts four. The same algorithm was therefore written directly with Qiskit's primitives. The core of the loop is shown below.

Excerpt from app/services/qaoa_service.py

```
op, ising_offset = to_ising(model.qubo)
scaled_op, scale = _normalised_cost_operator(op)      # divide by max |coefficient|
ansatz = QAOAAnsatz(cost_operator=scaled_op, reps=reps)
circuit = ansatz.decompose(reps=3)
circuit.measure_all()
sampler = StatevectorSampler(seed=seed, default_shots=shots)
def expectation(params):
    result = sampler.run([(circuit, np.asarray(params, dtype=float))]).result()
    counts = result[0].data.meas.get_counts()
    total = sum(counts.values())
    return sum(c * true_cost(b) for b, c in counts.items()) / total

x0 = np.concatenate([rng.uniform(0, np.pi, reps), rng.uniform(0, np.pi, reps)])
opt = minimize(expectation, x0, method="COBYLA",
               options={"maxiter": max(maxiter, ansatz.num_parameters + 2), "rhobeg":
0.5})
```

The function true_cost evaluates the original, unscaled QUBO on each bit string, so the values the optimizer minimises are in rupees. After optimization the circuit is sampled with four times the shots, and every distinct bit string is decoded and validated; the best feasible one is returned with the fraction of feasible shots, the probability of the returned string, the final angles and the time spent in each stage.

6.5 QUBO Construction

The compact encoding (function _build_compact) accumulates linear and quadratic coefficients in a builder object, in the order of Section 4.5: the opening cost of each store and the negated net margin of each link, the pairwise single-sourcing penalty for every two servers of a zone, the coupling penalty $\lambda x_{ij}(1 - y_j)$ for every link, and one product penalty for every minimal violating subset of each capacity constraint and of the budget.

The method penalise_product writes a product of one or two variables directly and chains longer products with AND-gadget variables, as in Equation 4.24. Subsets are enumerated up to size six for capacity and up to the number of stores for the budget.

6.6 Classical Baseline Implementation

The baseline states the identical model in PuLP with hard constraints, as in the excerpt below.

Excerpt from app/services/classical_service.py

```
prob = pulp.LpProblem("joint_darkstore_location_inventory", pulp.LpMaximize)
x = {s.id: pulp.LpVariable(f"x_{s.id}", cat=pulp.LpBinary) for s in instance.stores}
y = {(i, a): pulp.LpVariable(f"s_{i}__{a.replace(' ', '_')}", cat=pulp.LpBinary)
     for (i, a) in sorted(instance.links)}              # only service-feasible links
prob += (pulp.lpSum((instance.area(a).revenue - instance.delivery_cost[(i, a)]) * y[(i,
a)]
                    for (i, a) in y)
         - pulp.lpSum(instance.store(i).opening_cost * x[i] for i in x)), "net_margin"
for area in instance.areas:                             # C1
    servers = instance.servers_of(area.name)
    if servers:
        prob += pulp.lpSum(y[(i, area.name)] for i in servers) <= 1
for (i, a) in y:                                        # C2
    prob += y[(i, a)] <= x[i]
# C3 capacity and C4 budget use the same integerised data as the QUBO
solver = pulp.PULP_CBC_CMD(msg=False, timeLimit=time_limit_sec)
status_code = prob.solve(solver)
```

PuLP is pinned below version 3.0 because the 2.x series bundles the CBC binary, so no separate solver installation is needed.

6.7 Frontend Dashboard

The dashboard is a single-page React 19 application built with Vite and styled with Tailwind CSS. React Router provides six routes: the landing page, the optimizer, and the stores, products, demand and history pages. The optimizer form builds exactly the request body the backend expects, with budget, delivery limit, store, product and area selections and the QAOA options; storage capacity is shown read-only because it is a property of each store, not a request parameter. A preview modal shows the qubit count before a run. The results view shows the selected stores on a React-Leaflet map, with areas drawn as dashed approximate markers where their position is inferred from store centroids, and comparison cards and Recharts charts that place the QAOA and classical results side by side with their runtime, service level and optimality gap. The history page lists stored runs and opens their comparison.

6.8 Configuration and Deployment

All operating parameters are read from environment variables with documented defaults: DATABASE_URL, GROSS_MARGIN (0.18), RIDER_COST_PER_MIN (3.0), AVG_ITEMS_PER_ORDER (5.0), DEFAULT_DELIVERY_TIME_LIMIT (15 minutes), QAOA_MAX_QUBITS (20), QAOA_DEFAULT_REPS (3), QAOA_DEFAULT_MAXITER (200), QAOA_DEFAULT_SHOTS (2,048) and CLASSICAL_TIME_LIMIT_SEC (60). The backend runs with `uvicorn app.main:app` and the frontend with `npm run dev`.

CHAPTER 7
TESTING

7.1 Testing Strategy

Testing was carried out at four levels. Unit tests check individual services, such as the size and content of the QUBO, the penalty weight and the decoding of bit strings. API tests exercise every endpoint through FastAPI's test client, including the error paths. Constraint-validation tests give the validator plans with known violations and check that each violation is reported. Solver-comparison tests check that independent routes to the optimum agree: exhaustive enumeration of the QUBO, CBC and QAOA at sufficient depth.

The penalty terms need special attention because an error in one of them does not stop the program; it only gives a wrong answer that looks reasonable. For this reason the suite checks that every constraint is present in the reference model, that the penalty weight exceeds the best objective gain obtainable by cheating, and that the minimum of the QUBO found by enumerating all bit strings equals the proven optimum of the MILP. On a small instance this enumeration tests the whole encoding at once.

7.2 Test Environment

The suite uses pytest with FastAPI's TestClient. It runs against a throw-away SQLite database created in a temporary directory and seeded with the demo data, so it needs no database credentials and never touches the Supabase database; the same SQLAlchemy models, routers and services are exercised as in production. The tests were re-run for this report in a fresh virtual environment built from backend/requirements.txt on Python 3.14.7 under Windows 11.

7.3 Test Suite Summary

The automated suite contains 60 tests in four files, summarised in Table 7.1.

Table 7.1 Automated test suite of the backend

Test file	Tests	Areas covered
tests/test_database.py	4	Connectivity; all tables exist; seed loaded; every seed row declares its provenance
tests/test_crud_api.py	19	Store, product and demand CRUD; filters; 404 responses; rejection of negative cost, capacity and demand, zero volume, invalid latitude, zero delivery limit and unknown product; atomic bulk upload
tests/test_optimization_api.py	14	Root, health and documentation; preview qubit count; QAOA, classical and combined runs; low-depth QAOA still valid; run listing and comparison; unknown run; rejection of negative budget, unknown ids, budget below cheapest store and oversized QAOA instance

Test file	Tests	Areas covered
tests/test_solvers.py	23	QUBO size and content; penalty weight; brute-force optimum; CBC optimality, budget and capacity; QAOA feasibility, optimum at p = 3, behaviour at p = 1, rescaling and size limit; agreement of QAOA and CBC; validator checks; decoding

7.4 Solver and Constraint-Validation Tests

The 23 tests in tests/test_solvers.py are the core of the verification. Five of them check the QUBO: the compact encoding of the 3×3×3 instance has 11 qubits, the slack encoding is larger, all four constraint groups are present, the penalty weight exceeds the best gain from cheating, and the brute-force minimum equals the known optimum. Four check CBC: it proves the optimum, respects budget and capacity, and changes the plan when the budget is tightened. Seven check QAOA: it returns a feasible plan, reaches the optimum at p = 3, stays feasible but may be sub-optimal at p = 1, never claims proven optimality, records the Hamiltonian rescaling, refuses an oversized instance and agrees with CBC at sufficient depth. The remaining seven check the validator and the decoder: each of C1, C2, C3 and C4 violations is detected, an unknown store id is rejected, an empty plan is feasible with zero service, and auxiliary bits do not change the decoded plan.

7.5 Test Execution Results

The complete suite was executed with `python -m pytest tests -q`. All 60 tests passed in 17.30 seconds. The run produced 19 warnings, all of which are deprecation notices from Qiskit 2.x about classes that will change in Qiskit 3.0; none of them affects the results. The known optimum of ₹14,716.14 used by the tests is not injected into any solver; it is used only to assert that the independent routes agree with each other.

7.6 Design-Level Test Cases

The test cases planned at the design stage are listed in Table 7.2 together with the way each is covered in the implementation. Cases that concern features which were not implemented are marked accordingly.

Table 7.2 Design-level test cases and their coverage

ID	Test case	Expected output	Coverage in the implementation
T1	Data loading	48 sites and 163 zones loaded	Ingestion run: 48 stores, 163 zones, 25 products, 1,150 demand rows in the database
T2	Travel time matrix	One value for each store-zone pair	7,824 legs stored; 0 straight-line fallbacks
T3	Invalid records	Rejected during cleaning	20 of 68 fetched outlets rejected and logged by the ingestion step

ID	Test case	Expected output	Coverage in the implementation
T4	Single-sourcing penalty	Zero for a valid plan, positive for double assignment	test_validator_catches_double_assignment; test_qubo_has_all_constraints
T5	Assignment to closed store	Violation reported	test_validator_catches_inventory_in_unopened_store
T6	Capacity	Violation reported	test_validator_catches_capacity_overflow; test_classical_respects_capacity
T7	Budget	Violation reported	test_validator_catches_budget_violation; test_classical_respects_budget
T8	Encoding correctness	QUBO minimum equals MILP optimum	test_brute_force_matches_known_optimum
T9	QAOA run	Bit string decoded into a plan	test_qaoa_runs_and_is_feasible; test_decode_ignores_auxiliary_bits
T10	MILP baseline	Proven optimum returned	test_classical_finds_proven_optimum
T11	Validator	Known violation reported	Validator tests for C1 to C4 and unknown store ids (Section 7.4)
T12	Fast path	Order matched to nearest store with stock	Not applicable: fast path not implemented
T13	Slow path	Model built, solved, compared and stored	test_run_both_and_comparison; test_runs_are_listed

7.7 Checks on the Experimental Solutions

Beyond the automated suite, the solutions of the experiments in Chapter 8 were checked by the validator. Table 7.3 lists the checks for the prototype instance (Experiment A) and the OpenStreetMap instance (Experiment C).

Table 7.3 Validator checks on the experimental solutions

Check	Expected	Experiment A	Experiment C
Constraint violations (C1 to C5)	0	0	0
Service level	Reported	100%	50% (two zones unreachable)
Storage utilisation	≤ 100%	73.9%	14.5%
Budget utilisation	≤ 100%	85.0%	46.0%
QAOA objective vs proven optimum	Not higher	Equal (₹14,716.14)	Equal (₹100,704.01)
Brute-force QUBO minimum	Equals optimum	2,048 states, equal	32,768 states, equal

The comparison with the proven optimum is the most important check. Since CBC is exact, a feasible QAOA plan can never have a higher net margin; a higher value would reveal a broken constraint or a wrong penalty. In both experiments the values were equal, which is the best result that can be expected, and the brute-force minimum of the QUBO confirmed the encoding independently of both solvers.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Experimental Setup

All quantum results were obtained on Qiskit's noiseless statevector sampler; no experiment was run on quantum hardware. Unless stated otherwise, QAOA used depth p = 3, at most 150 COBYLA iterations, 2,048 shots per evaluation, 8,192 shots for the final sampling and random seed 42, with the compact encoding. CBC was run through PuLP with a time limit. The delivery-time promise was 15 minutes and the budget ₹20,000 unless stated otherwise.

Two kinds of figures are reported and are kept apart. Recorded values come from the project's own records: the backend README (prototype runs on the development machine) and the optimization tables of the local database (the OpenStreetMap run). Re-run values were produced while preparing this report, by executing the project's unmodified solver code on the same instances on the machine described in Table 3.4, using temporary copies of the data so that no project database was changed. Objective values are deterministic for a given seed and agreed exactly between recorded and re-run results; run times differ between machines and runs, and are therefore always labelled.

8.2 Overview of the Experiments

The results belong to different instances and must not be combined. Table 8.1 lists them.

Table 8.1 Experiments reported in this chapter

ID	Instance	Data	Qubits (compact)	Source of the result
A	Prototype: 3 stores × 3 areas × 3 products	Demo values; straight-line travel estimates	11	Recorded (README) and re-run
B	Supabase-backed instance	Supabase PostgreSQL data	Not recorded locally	Reported by the project team
C	4 OSM outlets × 4 zones × 25 products	OSM, OSRM, catalogue, population proxy	15	Recorded (local database) and re-run
D	Full stored network: 48 outlets × 46 demand zones × 25 products	As C	≥ 1,104 (not simulable)	Re-run, classical only
E	Exploratory scaling study, 3×3 to 6×7	Synthetic instances of the earlier development model	8 to 25	Recorded (stage5_scaling.csv)

8.3 Experiment A: Prototype Instance

On the prototype instance described in Section 4.6, QAOA with p = 3 and the PuLP/CBC baseline reached the same net margin of ₹14,716.14. CBC reported a proven optimum, and the exhaustive enumeration of all 2,048 bit strings of the 11-qubit QUBO gave the same

minimum, so the optimality gap of QAOA is 0% by Equation 4.37. The complete results are given in Table 8.2.

Table 8.2 Results of Experiment A (prototype instance)

Metric	QAOA (p = 3)	PuLP + CBC
Objective (net margin)	₹14,716.14	₹14,716.14 (proven optimal)
Optimality gap	0%	Reference
Stores opened	Venkata Nagar Hub 1, Lawspet Hub	Same
Margin / opening cost / rider cost	₹51,138.00 / ₹17,000.00 / ₹19,421.86	Same
Service level (demand units)	100% (6,800 of 6,800)	100%
Storage utilisation	73.9% (5,912.5 of 8,000)	73.9%
Budget utilisation	85.0% (₹17,000 of ₹20,000)	85.0%
Constraint violations	0	0
Qubits / circuit depth	11 / 71	10 binary variables, 14 constraints
Runtime, recorded (development machine)	1.38 s	0.019 s (separate API run, QAOA 2.03 s)
Runtime, re-run (Table 3.4)	7.31 s	0.058 s
Feasible fraction of final shots	4.2%	Not applicable

Figure 8.1 shows the two objective values side by side, and Figure 8.2 shows the service level and resource utilisation of the plan. The budget is the tighter of the two resources: opening the third store would exceed it, so the plan is limited by the budget rather than by storage. With 73.9% storage utilisation, 26.1% of the capacity of the opened stores is still free; with 85% budget utilisation, ₹3,000 of the budget is unspent.

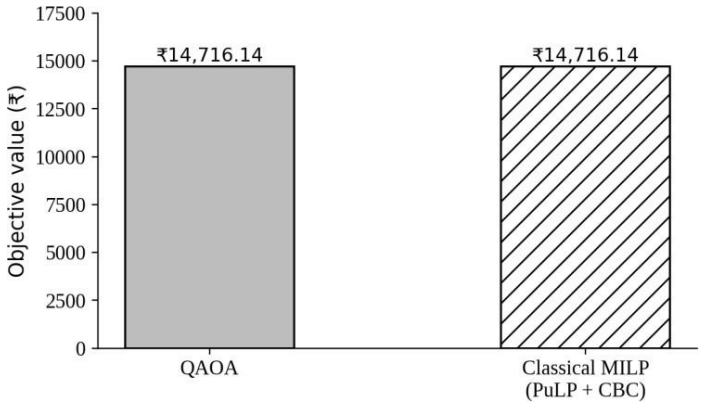


Figure 8.1 Objective value of QAOA and classical MILP on the prototype instance (Experiment A)

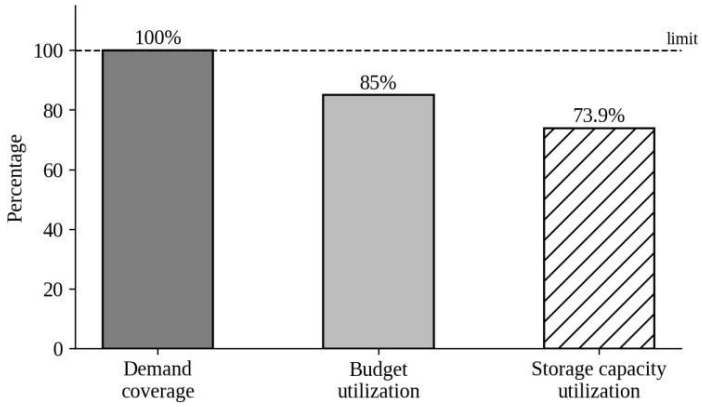


Figure 8.2 Service level and resource utilisation of the prototype solution (Experiment A)

The runtimes must be read with care. The recorded 1.38 s is the total QAOA time for the p = 3 run listed in the backend README; a separate API comparison run recorded 2.03 s for QAOA and 0.019 s for CBC. On the machine used for this report the same run took 7.31 s, of which 6.64 s was angle optimization. In every measurement CBC was faster by two orders of magnitude and, unlike QAOA, it provides a proof of optimality.

8.4 Sensitivity to Circuit Depth

QAOA is a sampling heuristic whose quality depends on its settings. Table 8.3 gives the re-run results on the prototype instance for the four settings documented in the project, and Figure 8.3 plots the objective values.

Table 8.3 Effect of circuit depth and optimizer budget on the prototype instance (re-run)

Setting	Circuit depth	Objective (₹)	Service level	Feasible shots	Runtime (s)
p = 1, 5 iterations, 256 shots	31	10,846.00	71.7%	25.4%	0.78
p = 1, 30 iterations, 512 shots	31	10,846.00	71.7%	27.4%	2.01
p = 2, 60 iterations, 1,024 shots	51	10,846.00	71.7%	24.9%	5.09
p = 3, 150 iterations, 2,048 shots	71	14,716.14	100%	4.2%	7.31

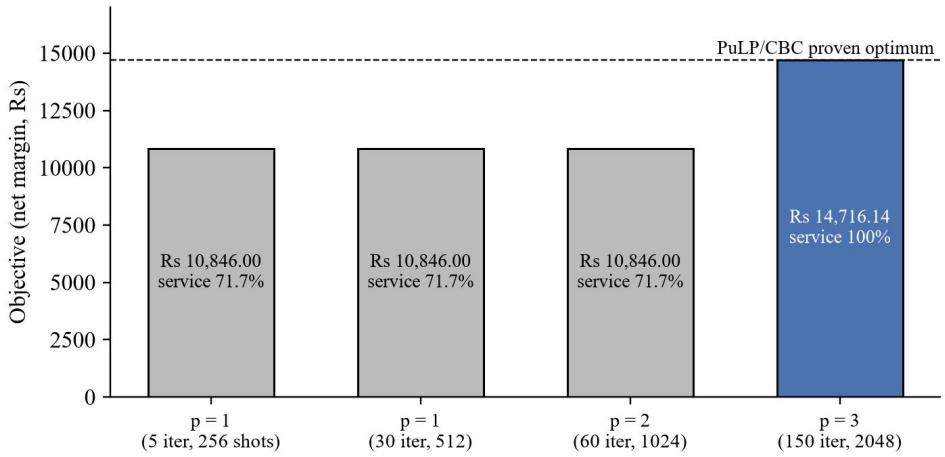


Figure 8.3 QAOA objective on the prototype instance for different depths and optimizer budgets

At p = 1 and p = 2 QAOA returned a feasible but sub-optimal plan. It opened the same two stores but left Marie Oulgaret unserved, losing that zone's margin of ₹14,517.00 less its rider cost of ₹10,646.86, so the net margin fell by ₹3,870.14 and the service level fell to 71.7%. Only at p = 3 did the best sampled plan reach the optimum. These figures also show that the feasible fraction of the final shots does not rise with depth: at p = 3 only 4.2% of the shots were feasible, yet one of them was optimal. This is typical of QAOA used as a sampler, where the result is the best feasible sample rather than the most probable one, and it shows that a statement about QAOA's quality needs repeated runs, not a single result.

8.5 Experiment B: Supabase-Backed Instance

After the database was moved to Supabase PostgreSQL, the project team recorded a run in which both QAOA and PuLP/CBC reached an objective value of ₹101,374.45. The full record of that run, including the selected stores, areas and products, the qubit count, the service level and the run times, is stored in the Supabase database and was not available in the local project files, so these details are not reported here and the run is not combined with any other experiment. The agreement of the two methods is consistent with the other experiments.

8.6 Experiment C: OpenStreetMap Instance with 25 Products

Experiment C uses real OpenStreetMap outlets and OSRM travel times: four candidate outlets (Pour Tous, Rathna Departmental Store, Green City and Nilgiri's, each with the prototype cost of ₹9,200 and capacity 4,200), four demand zones (Karuvadikuppam, Muthialpet, Veerampattinam and Villianur) and all 25 products. All sixteen store-zone legs are routed by OSRM. Only eight links meet the 15-minute promise, and none of them reaches Veerampattinam or Villianur. The QUBO has 12 decision variables and 3 AND-gadget variables, 15 qubits in total, against 79 with the slack encoding. Table 8.4 gives the results.

Table 8.4 Results of Experiment C (4 OSM outlets × 4 zones × 25 products)

Metric	QAOA (p = 3)	PuLP + CBC
Objective (net margin)	₹100,704.01	₹100,704.01 (proven optimal)
Optimality gap	0%	Reference
Store opened	Green City	Green City
Zones served	Karuvadikuppam (9.6 min), Muthialpet (7.2 min)	Same
Service level (demand units)	50.0% (1,140.44 of 2,280.88)	50.0%
Storage utilisation	14.5% (607.6 of 4,200)	14.5%
Budget utilisation	46.0% (₹9,200 of ₹20,000)	46.0%
Constraint violations	0	0
Runtime, recorded (local database)	18.58 s	0.087 s
Runtime, re-run (Table 3.4)	23.87 s	0.108 s
Feasible fraction of final shots	1.9%	Not applicable

The 50% service level is not a failure of either solver; it is a property of the instance. Veerampattinam and Villianur cannot be reached from any of the four outlets within 15 minutes, so the model is right not to serve them, and the C1 inequality allows this. Opening a second outlet would add ₹9,200 of cost without reaching a single extra zone, which is why only one store is opened even though the budget would permit two. The four zones have identical demand because each was allocated the same share of the municipal population by the proxy model, which is a visible consequence of using population instead of order data. The 100% coverage of Experiment A must therefore not be generalised: on real geography and a strict promise, coverage depends on where the candidate sites are.

The run times of Experiments A and C are compared in Figure 8.4. QAOA took about 24 s for 15 qubits on the re-run machine against 7.3 s for 11 qubits, while CBC stayed near 0.1 s; the angle optimization accounted for 22.8 s of the QAOA time in Experiment C.

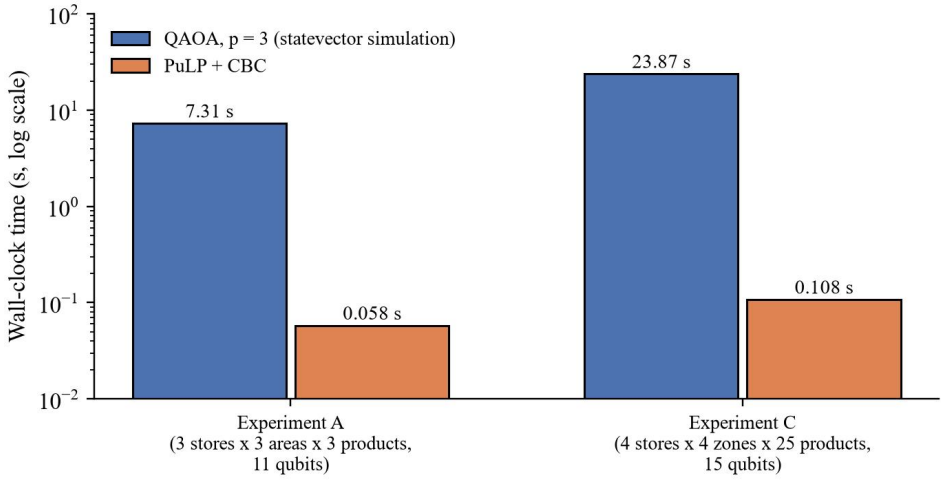


Figure 8.4 Re-run wall-clock time of QAOA and CBC in Experiments A and C (log scale)

8.7 Experiment D: Full Stored Network (Classical Only)

To check how the classical baseline behaves on the complete stored network, CBC was run while preparing this report on all 48 candidate outlets, the 46 zones that carry demand and all 25 products, with a 120-second time limit. Of the 2,208 store-zone pairs, 1,056 meet the 15-minute promise, giving 1,104 binary variables and 1,146 constraints; five zones (Ganapathichettikulam, Kilputhupattu, Korkadu, Sedarapet and Uruvaiyar) cannot be reached by any outlet within the promise. QAOA cannot be applied to this instance on a simulator, since at least 1,104 qubits would be needed. The results are in Table 8.5.

Table 8.5 Results of Experiment D (classical only, 48 outlets × 46 zones × 25 products)

Metric	Budget ₹20,000	Budget ₹100,000
CBC run time	120.2 s (time limit reached)	0.20 s
Status	Reported as Optimal by PuLP, but the limit was reached; treated as best found	Proven optimal
Objective (net margin)	₹1,475,725.67	₹2,205,629.54
Opening cost used	₹19,800	₹39,400
Service level	59.2%	89.0%
Storage utilisation	97.7%	70.1%

Two observations follow. First, the exact solver handles the complete stored network, but a tight budget makes the search much harder: with ₹100,000 the proof of optimality took 0.2 s, while with ₹20,000 CBC used its whole time limit. When the limit is reached, PuLP's status label alone does not establish that optimality was proven, so the value is treated here as the best solution found and no gap is computed against it. Second, the service level is limited by budget and geography together; even when the budget is not binding (₹39,400 of ₹100,000 used), about 11% of the demand remains unserved, including the five zones that no candidate outlet can reach within 15 minutes. These numbers depend on the synthetic store economics and the population proxy, and are reported to show solver behaviour, not to recommend a network.

8.8 Exploratory Scaling Study

Before the backend was built, the project ran a scaling study with its development scripts (stage5_scaling.py with qaoa_pipeline.py). That study used an earlier cost-minimisation version of the model on synthetic instances of increasing size, a NumPy statevector simulation written by hand, QAOA at $p = 3$ with three restarts and a single random seed, and compared QAOA with an exact enumeration, a greedy heuristic and simulated annealing. Its objective values are costs of a different model and cannot be compared with Experiments A to D; only the trends are of interest. Table 8.6 reproduces the recorded results.

Table 8.6 Recorded results of the exploratory scaling study (earlier model, synthetic instances)

Instance	Qubits	Exact cost	Greedy gap	Annealing gap	QAOA gap	QAOA feasible probability	QAOA time (s)
4×4 (stage 1)	14	23,413.49	0.0%	0.0%	20.8%	0.47	4.0
3×3	8	33,706.42	0.0%	0.0%	0.0%	0.69	0.6
3×4	10	30,723.05	0.0%	0.0%	0.0%	0.77	1.1
4×4	12	34,889.21	0.0%	0.0%	0.0%	0.56	1.5
4×5	13	48,565.40	0.0%	0.0%	5.5%	0.42	2.4
5×5	16	42,737.90	12.6%	0.0%	19.5%	0.38	43.5
5×6	19	66,920.72	5.5%	0.0%	12.9%	0.27	735.0
6×6	21	53,449.13	12.9%	0.0%	not run	not run	not run
6×7	25	57,353.88	4.7%	5.6%	not run	not run	not run

The study shows three trends. The qubit count grows steadily with the number of stores and areas. QAOA at a fixed depth matched the exact optimum on the smallest instances but fell behind as the instances grew, with gaps between 5% and 21%, and its probability of sampling a feasible plan fell from 0.77 to 0.27. The simulation time grew from under a second to more than twelve minutes at 19 qubits, which is why the study stopped there. Simulated annealing, a classical heuristic, matched the optimum on all but the largest instance. These observations motivated the compact encoding used in the backend and support the conclusion that, at the sizes that can be simulated, QAOA is a subject of study rather than a competitive solver.

8.9 Qubit Requirements and Simulation Memory

A statevector simulator stores 2^n complex amplitudes of 16 bytes each, so the memory it needs is given by Equation 8.1.

$$M = 16 \times 2^n \text{ bytes} \tag{8.1}$$

For 20 qubits this is 16 MiB and for 30 qubits 16 GiB, and every added qubit doubles it. Table 8.7 relates this to the instances of this study, and Figure 8.5 shows how the number of decision variables would grow with the number of zones if every zone could reach every one of the 48 stores, which is the upper bound $48 + 48 \times |I|$ of Equation 4.27.

Table 8.7 Qubits and statevector memory for the instances of this study

Instance	Qubits n	Memory 16×2^n bytes	Remark
Experiment A (compact)	11	32 KiB	Simulated in seconds
Experiment C (compact)	15	512 KiB	Simulated in about 20 s
Backend limit	20	16 MiB	Default QAOA_MAX_QUBITS
Experiment A (slack encoding)	62	64 EiB	Not representable
Experiment D (decision variables only)	1,104	Not representable	Classical only

Instance	Qubits n	Memory 16×2^n bytes	Remark
Full grid, 48 stores × 163 zones	7,872	Not representable	Upper bound if all links were feasible

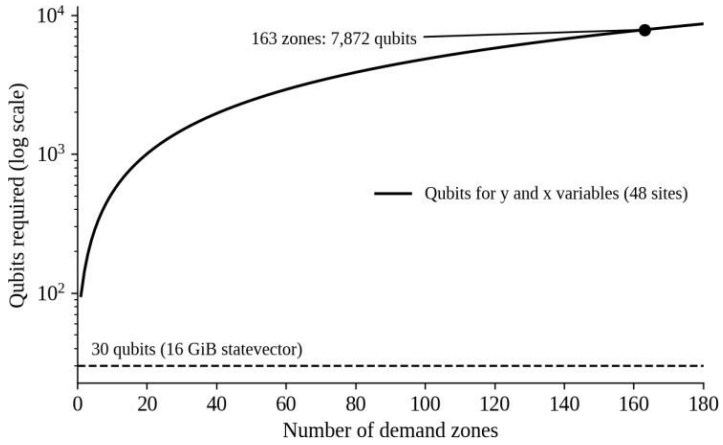


Figure 8.5 Decision variables against the number of demand zones for 48 candidate stores (upper bound, all links feasible)

The table explains the main limitation of the quantum branch. The compact encoding reduces the prototype from 62 to 11 qubits and makes simulation practical, but the complete network lies many orders of magnitude beyond any simulator and beyond the devices available today. Clustering the city into small sub-networks and further compression of the encoding are therefore necessary for larger quantum experiments.

8.10 Comparison of the Classical and Quantum Approaches

The comparison of the two approaches across all experiments is summarised in Table 8.8.

Table 8.8 Comparison of the classical MILP and the QAOA approach

Criterion	Classical MILP (PuLP + CBC)	QAOA (statevector simulator)
Objective on Experiments A and C	Proven optimum: ₹14,716.14 and ₹100,704.01	Same values at p = 3
Experiment B (team record)	₹101,374.45	₹101,374.45
Lower depth (Experiment A)	Not applicable	p = 1, 2: ₹10,846.00, feasible but sub-optimal
Largest instance solved	1,104 variables (Experiment D)	15 qubits (Experiment C)
Runtime	0.02 to 0.11 s on A and C	1.4 to 24 s on A and C, depending on machine
Optimality guarantee	Yes, when status Optimal within the limit	None; best feasible sample
Constraint handling	Hard linear constraints	Penalties and auxiliary variables; validation required
Noise	Not applicable	Not modelled

8.11 Comparison with Earlier Studies

Table 8.9 places the present work next to a few of the studies of Table 2.1 in terms of the problem size handled.

Table 8.9 Problem sizes in earlier studies and in the present work

Study	Problem size reported	Execution
Sinha (2026)	30 items, 5 locations, synthetic data	QAOA in Qiskit, compared with PuLP
Tata Elxsi pilot (2025)	5 parts selected out of 30	Native IBM quantum hardware
Bhattacharyya et al. (2022)	Performance drops beyond single-digit supply nodes	QAOA through the Qiskit interface
Present work	48 candidate outlets, 163 zones (46 with demand), 25 products, 7,824 routed legs	CBC on the full stored network; QAOA on 11- and 15-qubit instances on a noiseless simulator

8.12 Discussion

The experiments support four conclusions. First, the joint model is correct: on every instance small enough to check, the QUBO minimum found by enumeration, the CBC optimum and the best QAOA sample at p = 3 coincided, and the validator found no violation. Second, QAOA can reach the optimum of a small joint location-inventory instance, but its result depends on the depth and the optimizer budget, and lower settings return feasible but sub-optimal plans. Third, the compact encoding is what makes the quantum experiments possible at all, reducing the register from 62 to 11 qubits on the prototype and from 79 to 15 on the OpenStreetMap instance. Fourth, at every size tested the classical solver was faster by two orders of magnitude, handled instances far beyond the quantum branch and provided a proof of optimality.

These results do not show that QAOA is faster, more accurate or otherwise better than classical optimization, and no such claim is made. Each QAOA result is a single seeded run, so its variability across seeds was not measured, and the simulations exclude the noise of real devices, which the literature reports to degrade QAOA sharply on larger instances [23]. The value of the work lies in the formulation, the compact encoding, the independent validation and a working platform in which the quantum and classical methods are compared on identical, provenance-labelled data.

8.13 Limitations

- **Demand.** Demand is a population-derived estimate, not observed orders; only one of the 163 zones has a population figure sourced for that exact place.

- **Candidate stores.** The 48 outlets are retail shops mapped in OpenStreetMap and used as candidate sites; they are not verified dark stores, since operators do not tag dark stores in OpenStreetMap.
- **Store economics.** Opening costs, capacities, rider cost, handling time and gross margin are prototype assumptions; the prototype instance (Experiment A) also uses illustrative demand, prices and straight-line travel estimates.
- **Simulator.** All QAOA runs are noiseless statevector simulations limited to about 20 qubits; no hardware was used.
- **Repetitions.** Each reported QAOA result is a single seeded run; variability across seeds and instances was not measured.
- **Model scope.** Single period, single sourcing, inventory derived from assignment (no splitting of a zone's demand, no safety stock).
- **Solver status.** When CBC reaches its time limit, PuLP's status label can read Optimal although optimality was not proven, as in Experiment D with a ₹20,000 budget.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project proposed and implemented an integrated planning model for quick-commerce dark stores that decides store opening, the assignment of demand zones to stores and the resulting product-level inventory together, instead of one after the other. The conceptual cost-minimisation formulation of the conference paper was refined into an implemented model that maximises net operating margin, enforces single sourcing, store-assignment coupling, storage capacity and the opening budget as constraints, enforces the delivery-time promise structurally through the set of service-feasible links, and derives inventory exactly from the assignment.

The model was encoded as a QUBO with a compact encoding based on minimal violating subsets and AND gadgets, which reduced the register from 62 to 11 qubits on the prototype and from 79 to 15 qubits on an OpenStreetMap instance, converted to an Ising Hamiltonian and solved with QAOA on a Qiskit statevector simulator. Every instance was also solved exactly with PuLP and CBC, and every solution was checked by an independent validator. The model was embedded in a working platform with an open-data ingestion pipeline, a FastAPI backend, a Supabase PostgreSQL database and a React dashboard, covered by 60 automated tests that all pass.

On the prototype instance, QAOA at depth three reached the proven optimum of ₹14,716.14 with full service and no violation, while depths one and two returned feasible plans worth ₹10,846.00. On the OpenStreetMap instance with 25 products both methods reached ₹100,704.01 with 50% service, limited by the delivery promise, and a later Supabase-backed run recorded ₹101,374.45 for both. CBC was faster by two orders of magnitude in every case and solved the complete stored network of 1,104 variables, far beyond the reach of the quantum branch. The work therefore demonstrates the correctness and executability of a joint location-inventory QUBO on a gate-based circuit model, and makes no claim of quantum advantage. Its limitations, listed in Section 8.13, concern the demand proxy, the unverified candidate stores and synthetic store economics, the noiseless simulator and the single seeded runs.

9.2 Contributions

- A joint formulation in which siting, assignment and inventory share one set of binary variables, with inventory derived rather than searched.

- A compact QUBO encoding of the capacity and budget inequalities that avoids slack qubits, with an instance-derived penalty weight.
- A QAOA implementation on Qiskit primitives with Hamiltonian normalisation, true-cost expectation and best-feasible-sample selection.
- An independent validator and a comparison service that publish an optimality gap only against a proven optimum.
- A provenance-labelled open-data pipeline for Puducherry and a web platform that places the quantum and classical results side by side.

9.3 Future Scope

- **Better demand data.** Replace the population proxy with authorised order histories and add demand forecasting, using the historical-orders mode for which the pipeline is already structured; obtain ward-level census data where available.
- **Verified candidate stores and economics.** Use operator-provided dark-store locations, lease costs, capacities and rider costs in place of the prototype assumptions.
- **Larger optimization instances.** Decompose the city into clusters of stores and zones that fit within a simulator or device, and investigate tighter encodings and constraint-preserving mixers [11].
- **Repeated experiments.** Run each QAOA configuration over many seeds and instances, and report the distribution of objective values, feasible-shot fractions and run times rather than single runs.
- **Additional datasets.** Evaluate the framework on other cities and on standard facility-location benchmarks to test whether the conclusions generalise.
- **Noise and hardware.** Add noise models, transpile the circuits to real device topologies and test small instances on quantum hardware with error mitigation.
- **Model extensions.** Extend to multiple periods, split sourcing, safety stock and replenishment, and implement the live-order fast path of the original design.
- **Deployment hardening.** Add authentication, restrict CORS and run the test suite against PostgreSQL as well as SQLite.

REFERENCES

[1] Agrawal, P., Balkanski, E., Gkatzelis, V., Ou, T., and Tan, X., "Learning-Augmented Mechanism Design: Leveraging Predictions for Facility Location," *Mathematics of Operations Research*, Vol. 49, 2024, pp. 2626–2651.

[2] Amin-Tahmasbi, H., Sadafi, S., Ekren, B. Y., and Kumar, V., "A Multi-Objective Integrated Optimisation Model for Facility Location and Order Allocation Problem in a Two-Level Supply Chain Network," *Annals of Operations Research*, Vol. 324, 2023, pp. 993–1022.

[3] Azad, U., Behera, B. K., Ahmed, E. A., Panigrahi, P. K., and Farouk, A., "Solving Vehicle Routing Problem Using Quantum Approximate Optimization Algorithm," *IEEE Transactions on Intelligent Transportation Systems*, Vol. 24, 2023, pp. 7564–7573.

[4] Boros, E., and Hammer, P. L., "Pseudo-Boolean Optimization," *Discrete Applied Mathematics*, Vol. 123, 2002, pp. 155–225.

[5] Church, R., and ReVelle, C., "The Maximal Covering Location Problem," *Papers of the Regional Science Association*, Vol. 32, 1974, pp. 101–118.

[6] Daskin, M. S., Coullard, C. R., and Shen, Z.-J. M., "An Inventory-Location Model: Formulation, Solution Algorithm and Computational Results," *Annals of Operations Research*, Vol. 110, 2002, pp. 83–106.

[7] Farhi, E., Goldstone, J., and Gutmann, S., "A Quantum Approximate Optimization Algorithm," arXiv preprint arXiv:1411.4028, 2014.

[8] Fitzek, D., Ghandriz, T., Laine, L., Granath, M., and Kockum, A. F., "Applying Quantum Approximate Optimization to the Heterogeneous Vehicle Routing Problem," *Scientific Reports*, Vol. 14, 2024, Article 25415.

[9] Glover, F., Kochenberger, G., and Du, Y., "Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models," *4OR*, Vol. 17, 2019, pp. 335–371.

[10] Gomes, C., Fernandes, J. P., Falcão, G., Kar, S., and Tayur, S., "A Systematic Mapping Study on Quantum and Quantum-inspired Algorithms in Operations Research," *ACM Computing Surveys*, Vol. 57, 2025, pp. 1–35.

[11] Hadfield, S., Wang, Z., O'Gorman, B., Rieffel, E. G., Venturelli, D., and Biswas, R., "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz," *Algorithms*, Vol. 12, No. 2, 2019, Article 34.

[12] Haklay, M., and Weber, P., "OpenStreetMap: User-Generated Street Maps," *IEEE Pervasive Computing*, Vol. 7, No. 4, 2008, pp. 12–18.

[13] Harris, F. W., "How Many Parts to Make at Once," *Factory, The Magazine of Management*, Vol. 10, No. 2, 1913, pp. 135–136.

[14] Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., and Gambetta, J. M., "Quantum Computing with Qiskit," arXiv preprint arXiv:2405.08810, 2024.

[15] Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P., "Optimization by Simulated Annealing," *Science*, Vol. 220, No. 4598, 1983, pp. 671–680.

[16] Land, A. H., and Doig, A. G., "An Automatic Method of Solving Discrete Programming Problems," *Econometrica*, Vol. 28, No. 3, 1960, pp. 497–520.

[17] Lucas, A., "Ising Formulations of Many NP Problems," *Frontiers in Physics*, Vol. 2, 2014, Article 5.

[18] Luxen, D., and Vetter, C., "Real-Time Routing with OpenStreetMap Data," in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011, pp. 513–516.

[19] Melo, M. T., Nickel, S., and Saldanha-da-Gama, F., "Facility Location and Supply Chain Management – A Review," *European Journal of Operational Research*, Vol. 196, No. 2, 2009, pp. 401–412.

[20] Mitchell, S., O'Sullivan, M., and Dunning, I., "PuLP: A Linear Programming Toolkit for Python," The University of Auckland, Auckland, New Zealand, 2011.

[21] Moncayo-Martínez, L. A., and He, N., "Quantum Optimisation for Supply Chain: QUBO Formulations and QAOA Solutions for Facility Location and Load Balancing," *Results in Engineering*, Vol. 29, 2026, Article 108373.

[22] Owen, S. H., and Daskin, M. S., "Strategic Facility Location: A Review," *European Journal of Operational Research*, Vol. 111, No. 3, 1998, pp. 423–447.

[23] Pellow-Jarman, A., McFarthing, S., Sinayskiy, I., Park, D. K., Pillay, A., and Petruccione, F., "The Effect of Classical Optimizers and Ansatz Depth on QAOA Performance in Noisy Devices," *Scientific Reports*, Vol. 14, 2024, Article 16011.

[24] Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., Aspuru-Guzik, A., and O'Brien, J. L., "A Variational Eigenvalue Solver on a Photonic Quantum Processor," *Nature Communications*, Vol. 5, 2014, Article 4213.

[25] Powell, M. J. D., "A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation," in Gomez, S., and Hennart, J.-P. (Eds.), *Advances in Optimization and Numerical Analysis*, Kluwer Academic, Dordrecht, 1994, pp. 51–67.

[26] Preskill, J., "Quantum Computing in the NISQ Era and Beyond," *Quantum*, Vol. 2, 2018, Article 79.

[27] Raj, A., Das, D., and Sawik, T., "Mitigating Disruption Impact in Q-Commerce Through Optimization of Dark Store Resilient Portfolio," *Transportation Research Part E: Logistics and Transportation Review*, Vol. 205, 2026, Article 104518.

[28] Sanghi, N., Balodi, K. C., and Gupta, V., "The Emergence of the Indian Hyperlocal Grocery Delivery Industry: Dunzo v/s Blinkit," *Journal of Information Technology Teaching Cases*, Vol. 14, 2024, pp. 119–128.

[29] Shen, Z.-J. M., Coullard, C., and Daskin, M. S., "A Joint Location-Inventory Model," *Transportation Science*, Vol. 37, No. 1, 2003, pp. 40–55.

[30] Yang, X., Ostermeier, M., and Hübner, A., "Winning the Race to Customers with Micro-Fulfillment Centers: An Approach for Network Planning in Quick Commerce," *Central European Journal of Operations Research*, Vol. 32, 2024, pp. 295–334.

[31] Zhou, L., Wang, S.-T., Choi, S., Pichler, H., and Lukin, M. D., "Quantum Approximate Optimization Algorithm: Performance, Mechanism, and Implementation on Near-Term Devices," *Physical Review X*, Vol. 10, 2020, Article 021067.